ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA HỆ THỐNG THÔNG TIN

BÁO CÁO ĐỒ ÁN

MÔN: MẠNG XÃ HỘI

ĐỀ TÀI:

ỨNG DỤNG GRAPH CONVOLUTIONAL NETWORK VÀO DỰ ĐOÁN LIÊN KẾT GIỮA CÁC PROTEIN

GVHD: Hà Lê Hoài Trung

Lớp: IS353.P21

Nhóm sinh viên thực hiện:

1. Vũ Minh Đức – 22520277
2. Nguyễn Hữu Thiện – 22521389
3. Phan Thị Tường Vi – 22521658
4. Nguyễn Thị Thúy Vy – 22521706

DANH MỤC BẢNG BIỂU, HÌNH ẢNH

Danh mục các bảng

Bảng 1 Các cột chính về ID & Tương tác 7
Bảng 2 Thông tin về Protein & Tên gọi 7
Bảng 3 Loại tương tác & Nguồn gốc 7
Bảng 4 Thông tin về sinh vật 7
Bảng 5 Các cột bổ sung 8
Bảng 6 Thông tin về cơ sở dữ liệu Protein 8
Bảng 7 Thông tin về thuật ngữ khoa học 8
Bảng 8 So sánh đặc trưng GCNs và GATs 19
Bảng 9 So sánh điểm mạnh GCNs và GATs 21
Bảng 10 So sánh điểm yếu của GCNs và GATs 22
Bảng 11 Đầu vào và kiến trúc của các phương pháp 31
Bảng 12 So sánh các chỉ số hiệu suất 32
Bảng 13 Thống kê kết quả evaluation cuối cùng 46
Bảng 14 Dự đoán tương tác 10 cặp protein ngẫu nhiên 49

Danh mục các hình ảnh

Hình 1 Năm dòng đầu của bộ dữ liệu 6
Hình 2 Trực quan phân bổ số lượng hệ thống thí nghiệm 9
Hình 3 Trực quan phân bổ Experimental System Type 10
Hình 4 Top 10 sinh vật A có số lượng tương tác cao nhất 10
Hình 5 Top 10 sinh vật B có số lượng tương tác cao nhất 11
Hình 6 Phân bổ của physical ratio 12
Hình 7 Phân bổ của genetic ratio 12
Hình 8 Mô hình hóa đồ thị tương tác của 10 nodes có nhiều liên kết nhất 13
Hình 9 Minh họa thông tin của 1 node cụ thể với ID: 4383847 với các đặc trưng ban đầu 14
Hình 10 Phân phối Eigenvector theo số lượng protein 23
Hình 11 Top 20 protein theo eigenvector 24
Hình 12 Phân phối protein theo Closeness Centrality 25
Hình 13 Top 20 protein theo Closeness Centrality 25
Hình 14 Phân phối protein theo Betweness Centrality 26
Hình 15 Top 20 protein theo Betweeness Centrality 27
Hình 16 Phân phối protein theo Clustering 28
Hình 17 Phân phối protein theo Triangle Count 29
Hình 18 Top 20 node có Triangle Count cao nhất 29
Hình 19 Sơ đồ hoá các bước chạy chương trình 34

Hình 20 Năm dòng đầu của dữ liệu đã được tiền xử lý 35
Hình 21 Phân phối các đặc trưng trước khi log_transform 36
Hình 22 Phân phối đặc trưng sau log_transform 38
Hình 23 Ma trận tương quan các đặc trưng đã tính 39
Hình 24 Ma trận tương quan sau khi loại bỏ các đặc trưng có tương quan cao 41
Hình 25 Đoạn mã minh hoạ việc tính toán các số cạnh 43
Hình 26 Loss và AUC của 50 epochs 45
Hình 27 Confusion matrix 47
Hình 28 Phân phối điểm số giữa cạnh dương và cạnh âm 48

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN 6
1.1. Bài toán cần giải quyết – Phân loại bài toán 6
1.1.1. Bài toán cần giải quyết 6
1.1.2. Phân loại bài toán 6
1.2. Giới thiệu bộ dữ liệu và mô tả các thuộc tính 6
1.2.1. Bộ dữ liệu 6
1.2.2. Các thuộc tính 6
1.2.3. Trực quan bộ dữ liệu 9
1.2.4. Trích xuất đặc trưng sinh học từ DataFrame 11
1.3. Mô hình bài toán 12
1.3.1. Nodes 12
1.3.2. Edges 13
1.3.3. Mô hình hóa đồ thị tương tác 13
1.3.4. Ma trận đặc trưng đầu vào 14
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 15
2.1. Giới thiệu tổng quan 15
2.2. Nguyên lý hoạt động 15
2.3. Ứng dụng của graph trong phân tích mạng xã hội 18
2.4. So sánh GCNs và GATs 18
2.4.1. Đặc trưng 19
2.4.2. Điểm mạnh 21
2.4.3. Điểm yếu 21
2.5. Các đặc trưng của graph 22
2.6. Bài báo về graph GCN 29
2.6.1. Giới thiệu bài báo 30
2.6.2. Mục tiêu nghiên cứu 30
2.6.3. Phương pháp thực hiện 30
2.6.4. Điểm mạnh 31

2.6.5. Điểm yếu 33
CHƯƠNG 3: HIỆN THỰC 34
3.1. Sơ đồ hóa các bước chạy chương trình 34
3.2. Giải thích các bước chạy chương trình 34
3.2.1. Tiền xử lý dữ liệu 34
3.2.2. Xây dựng đồ thị 35
3.2.3. Trích xuất đặc trưng 35
3.2.4. Lựa chọn đặc trưng 39
3.2.5. Ghép và chuẩn hóa đặc trưng 42
3.2.6. Tạo danh sách cạnh và chia tập dữ liệu 42
3.2.7. Xây dựng mô hình 43
3.2.8. Huấn luyện mô hình 44
3.2.9. Dự đoán liên kết 45
3.3. Trực quan kết quả 45
3.3.1. Loss và AUC 45
3.3.2. Kết quả evaluation và Confusion Matrix 46
3.3.3. Phân phối điểm số giữa cạnh dương và cạnh âm 47
3.3.4. Dự đoán tương tác 10 cặp protein ngẫu nhiên 48
TÀI LIỆU THAM KHẢO 50

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN

1.1. Bài toán cần giải quyết – Phân loại bài toán

1.1.1. Bài toán cần giải quyết

Sử dụng bộ dữ liệu BIOGRID-CORONAVIRUS, bài toán đặt ra là xây dựng và phân tích mạng lưới tương tác protein liên quan đến coronavirus bằng cách áp dụng Graph Convolutional Networks (GCNs).

- Xây dựng mạng lưới nơi mỗi đỉnh đại diện cho một protein, và mỗi cạnh biểu diễn tương tác giữa các protein.
- Áp dụng GCNs để học biểu diễn đặc trưng cho từng protein từ cấu trúc đồ thị và các thuộc tính sinh học liên quan.
- Dự đoán các tương tác protein tiềm năng chưa được phát hiện.
- Phân tích mạng để xác định các protein trung tâm hoặc có ảnh hưởng lớn.

1.1.2. Phân loại bài toán

- **Phân tích cấu trúc mạng (Graph Analysis):** Xây dựng và trực quan hóa mạng tương tác protein để tìm ra các cụm quan trọng.
- **Xác định nút quan trọng (Centrality Analysis):** Tìm các protein có vai trò trung tâm trong mạng tương tác.
- **Mô hình hóa và dự đoán tương tác (Link Prediction):** Dự đoán các tương tác mới có thể xảy ra dựa trên dữ liệu hiện có.

1.2. Giới thiệu bộ dữ liệu và mô tả các thuộc tính

1.2.1. Bộ dữ liệu

- **Nguồn bộ dữ liệu:** BIOGRID-4.4.245/BIOGRID-CORONAVIRUS
- **Tên bộ dữ liệu:** Biogrid-Coronavirus
- **Kích thước bộ dữ liệu:** 3.88 MB với 68578 dòng và 37 cột

```
df.head()
```

	#BioGRID Interaction ID	Entrez Gene Interactor A	Entrez Gene Interactor B	BioGRID ID Interactor A	BioGRID ID Interactor B	Systematic Name Interactor A	Systematic Name Interactor B	Official Symbol Interactor A	Official Symbol Interactor B	Synonyms Interactor A	...	TREMBL Accessions Interactor B	REFSEQ Accessions Interactor B	Ontology Term IDs	Ontology Term Names	Ontology Term Categories	Ontology Term Qualifier IDs	Ontology Term Qualifier Names	Ontology Term Types	Organism Name Interactor A	Organism Name Interactor B
0	141211	850980	852501	31542	32899	YLR274W	YBR202W	MCM5	MCM7	BOB1|CDC46|MCM DNA helicase complex subunit MC...	...	-	NP_009761	-	-	-	-	-	-	Saccharomyces cerevisiae (S288c)	Saccharomyces cerevisiae (S288c)
1	141212	852501	850980	32899	31542	YBR202W	YLR274W	MCM7	MCM5	CDC47|MCM DNA helicase complex subunit MCM7|L0...	...	-	NP_013376	-	-	-	-	-	-	Saccharomyces cerevisiae (S288c)	Saccharomyces cerevisiae (S288c)
2	141213	850980	856680	31542	36697	YLR274W	YEL032W	MCM5	MCM3	BOB1|CDC46|MCM DNA helicase complex subunit MC...	...	-	NP_010882	-	-	-	-	-	-	Saccharomyces cerevisiae (S288c)	Saccharomyces cerevisiae (S288c)
3	141214	856680	850980	36697	31542	YEL032W	YLR274W	MCM3	MCM5	MCM DNA helicase complex subunit MCM3|L000001039	...	-	NP_013376	-	-	-	-	-	-	Saccharomyces cerevisiae (S288c)	Saccharomyces cerevisiae (S288c)
4	141215	852501	850793	32899	31376	YBR202W	YLR103C	MCM7	CDC45	CDC47|MCM DNA helicase complex subunit MCM7|L0...	...	-	NP_013204	-	-	-	-	-	-	Saccharomyces cerevisiae (S288c)	Saccharomyces cerevisiae (S288c)

5 rows × 37 columns

Hình 1 Năm dòng đầu của bộ dữ liệu

1.2.2. Các thuộc tính

1.2.2.1. Các cột chính về ID & Tương tác

Bảng 1 Các cột chính về ID & Tương tác

Tên Cột	**Giải thích**
BioGRID Interaction ID	Mã số duy nhất cho mỗi tương tác protein-protein.
Entrez Gene Interactor A/B	ID của gen tương ứng với protein trong cơ sở dữ liệu Entrez Gene.
BioGRID ID Interactor A/B	ID của protein trong cơ sở dữ liệu BioGRID.

Mục đích: Dùng để xác định duy nhất mỗi protein.

1.2.2.2. Thông tin về Protein & Tên gọi

Bảng 2 Thông tin về Protein & Tên gọi

Tên Cột	**Giải thích**
Systematic Name Interactor A/B	Tên hệ thống (systematic name) của protein, thường là tên khoa học được đặt theo mã gen.
Official Symbol Interactor A/B	Tên chính thức của protein, được sử dụng rộng rãi trong nghiên cứu.
Synonyms Interactor A/B	Các tên khác (bí danh, alias) của protein này.

Mục đích: Dùng để tìm kiếm protein theo nhiều cách đặt tên khác nhau.

1.2.2.3. Loại tương tác & Nguồn gốc

Bảng 3 Loại tương tác & Nguồn gốc

Tên cột	**Giải thích**
Experimental System	Phương pháp thực nghiệm được sử dụng để xác định tương tác (VD: Two-hybrid, Co-immunoprecipitation).
Experimental System Type	Loại tương tác (physical - tương tác vật lý hoặc genetic - tương tác di truyền).
Author	Tác giả của nghiên cứu phát hiện tương tác này.
Publication Source	Nguồn xuất bản (thường là mã PubMed).

Mục đích: Giúp xác minh độ tin cậy của dữ liệu.

1.2.2.4. Thông tin về sinh vật

Bảng 4 Thông tin về sinh vật

Tên Cột	**Giải thích**
Organism ID Interactor A/B	ID của sinh vật chứa protein này (theo chuẩn NCBI Taxonomy).

Organism Name Interactor A/B	Tên khoa học của sinh vật chứa protein này.

Mục đích: Phân biệt giữa protein của các loài khác nhau (VD: virus corona và con người).

1.2.2.5. Các cột bổ sung

Bảng 5 Các cột bổ sung

Tên Cột	**Giải thích**
Throughput	Loại thí nghiệm (High Throughput - quy mô lớn, Low Throughput - quy mô nhỏ).
Score	Điểm số tin cậy của tương tác (có thể không có giá trị).
Modification	Sự sửa đổi protein liên quan đến tương tác này (nếu có).
Qualifications	Ghi chú thêm về tương tác này.
Tags	Thông tin bổ sung về thí nghiệm.
Source Database	Cơ sở dữ liệu gốc của tương tác này (VD: BioGRID).

Mục đích: Cung cấp thêm thông tin về chất lượng & bối cảnh của dữ liệu.

1.2.2.6. Thông tin về cơ sở dữ liệu Protein

Bảng 6 Thông tin về cơ sở dữ liệu Protein

Tên Cột	**Giải thích**
SWISS-PROT Accessions Interactor A/B	ID protein trong cơ sở dữ liệu SWISS-PROT (UniProtKB).
TREMBL Accessions Interactor A/B	ID protein trong UniProtKB/TrEMBL.
REFSEQ Accessions Interactor A/B	ID protein trong cơ sở dữ liệu RefSeq (NCBI).

Mục đích: Cho phép đối chiếu với các cơ sở dữ liệu sinh học khác.

1.2.2.7. Thông tin về thuật ngữ khoa học

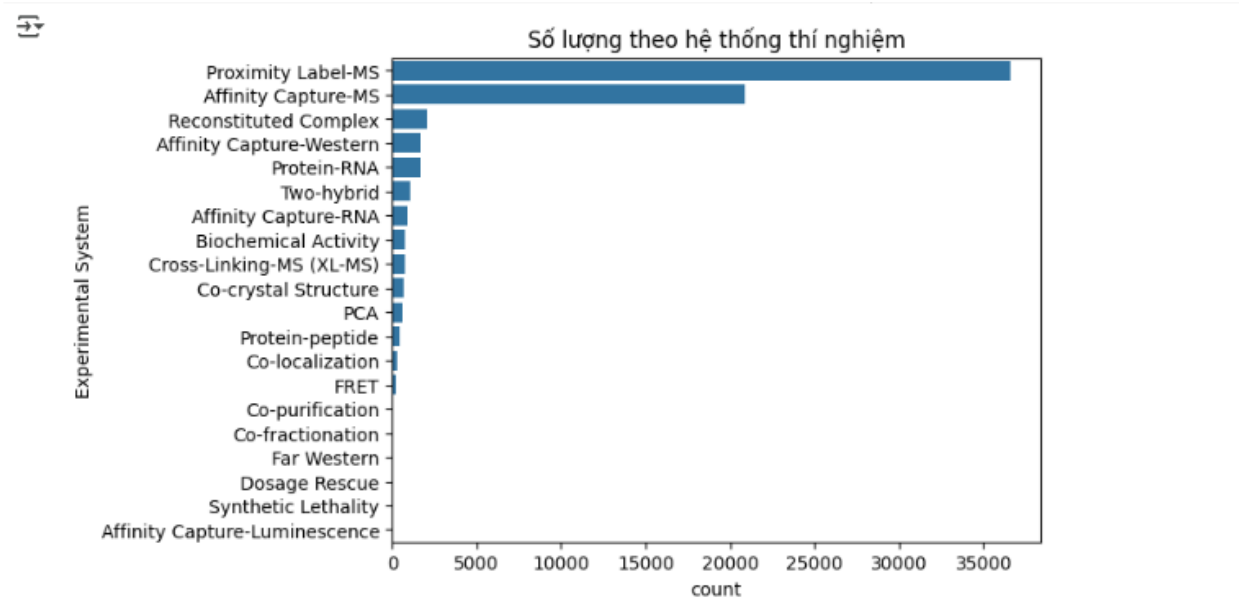
Bảng 7 Thông tin về thuật ngữ khoa học

Tên Cột	**Giải thích**
Ontology Term IDs	Mã ID của thuật ngữ sinh học liên quan đến tương tác này.
Ontology Term Names	Tên thuật ngữ sinh học tương ứng.
Ontology Term Categories	Nhóm thuật ngữ sinh học (VD: Molecular Function, Biological Process).

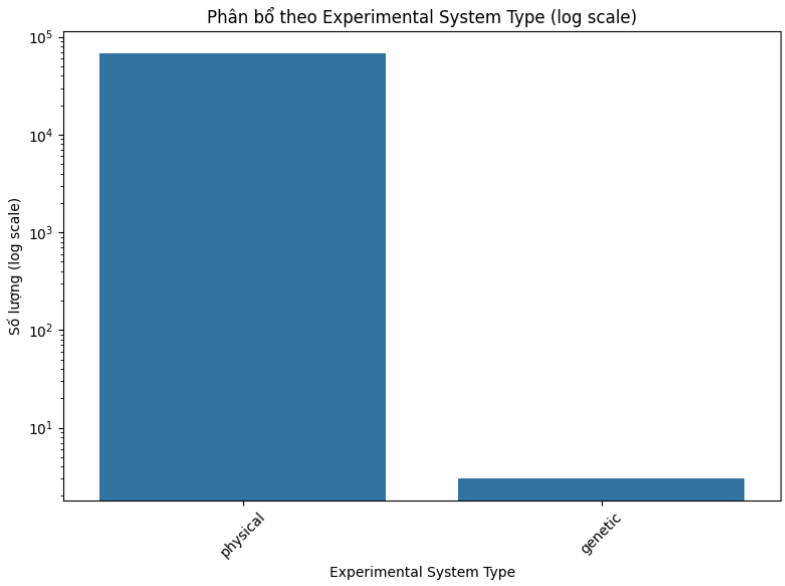
Ontology Term Qualifier IDs	ID của thông tin bổ sung về thuật ngữ.
Ontology Term Qualifier Names	Tên thông tin bổ sung.
Ontology Term Types	Loại thuật ngữ (VD: GO - Gene Ontology).

Mục đích: Giúp phân loại các chức năng sinh học liên quan đến tương tác.

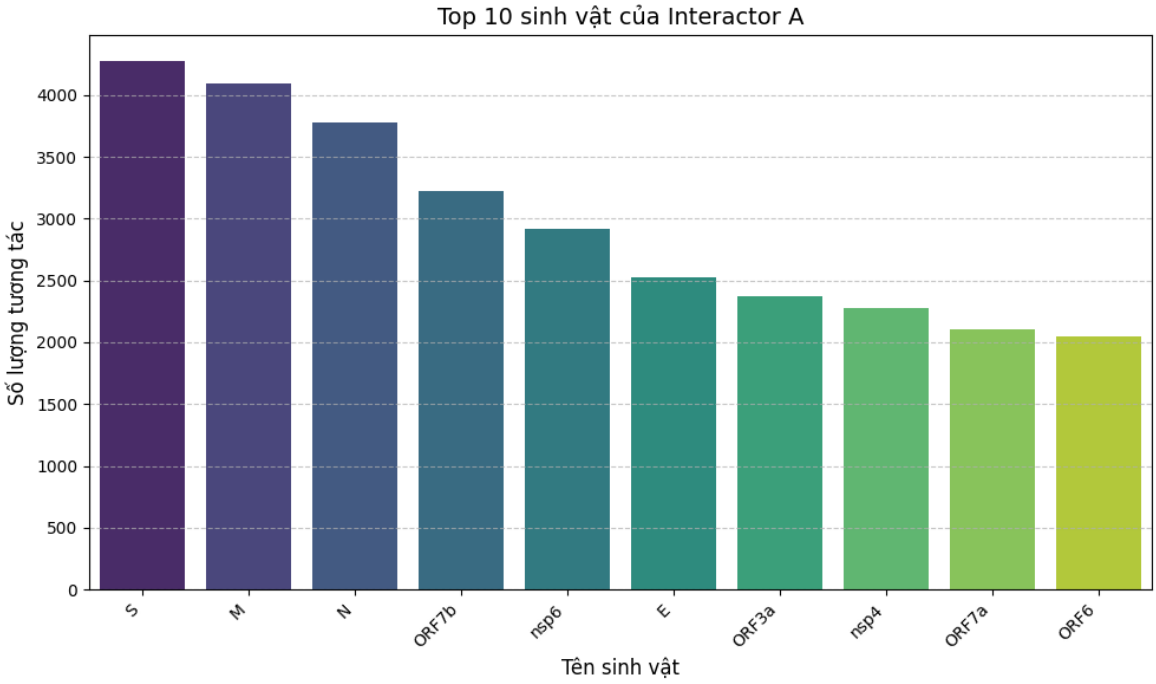
1.2.3. Trực quan bộ dữ liệu



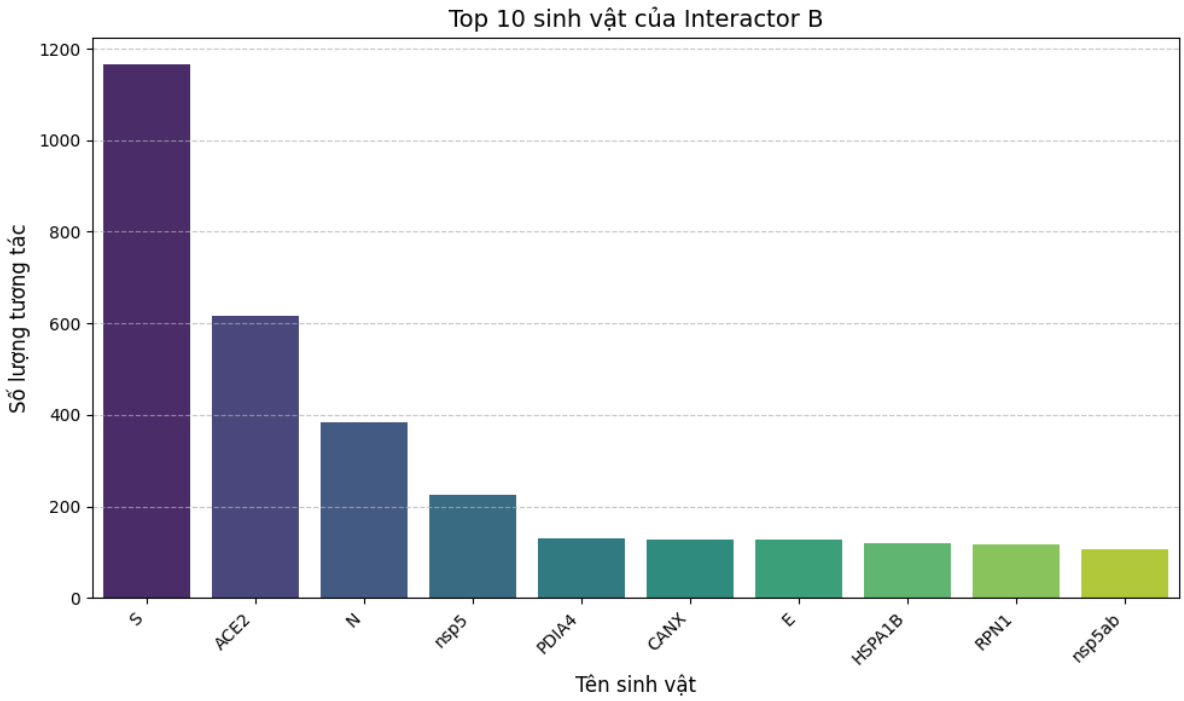
Hình 2 Trực quan phân bổ số lượng hệ thống thí nghiệm



Hình 3 Trực quan phân bổ Experimental System Type



Hình 4 Top 10 sinh vật A có số lượng tương tác cao nhất



Hình 5 Top 10 sinh vật B có số lượng tương tác cao nhất

1.2.4. Trích xuất đặc trưng sinh học từ DataFrame

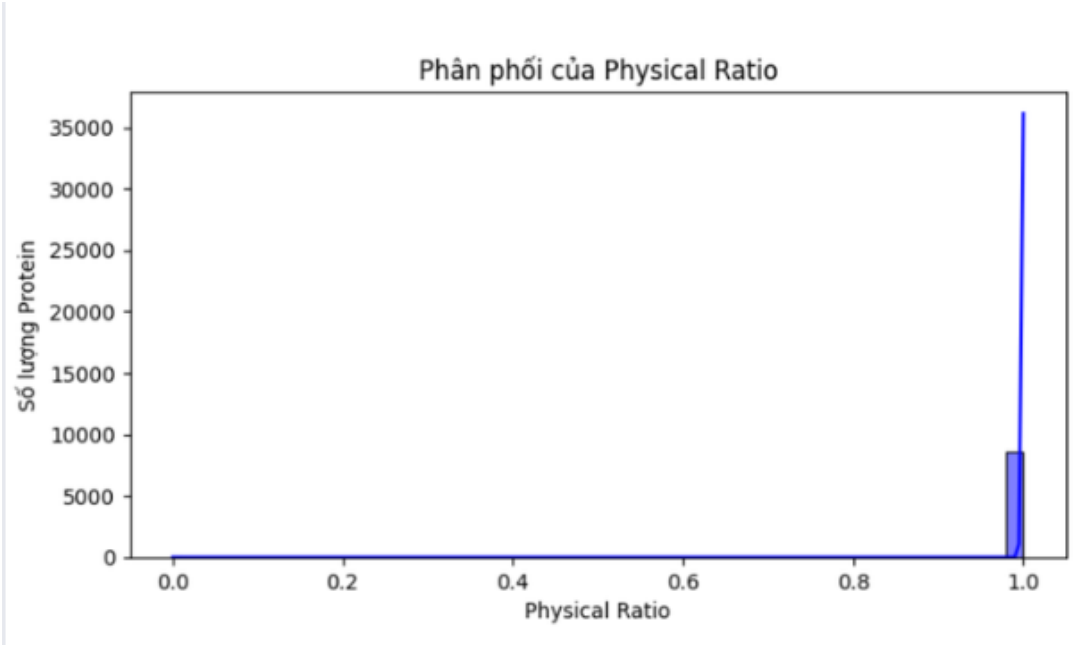
Trích xuất 2 đặc trưng sinh học gồm:

- **Physical_ratio**: Tỷ lệ tương tác vật lý so với tổng số tương tác của protein đó.
- **Generic_ratio**: Tỷ lệ tương tác di truyền so với tổng số tương tác của protein đó.

Để áp dụng các kỹ thuật học máy cho bài toán dự đoán liên kết protein, ta cần chuyển dữ liệu từ BioGRID thành các đặc trưng số có thể sử dụng trong mô hình học máy. Vì dữ liệu gốc không có các đặc trưng số rõ ràng nên cần phải có bước chuyển đổi.

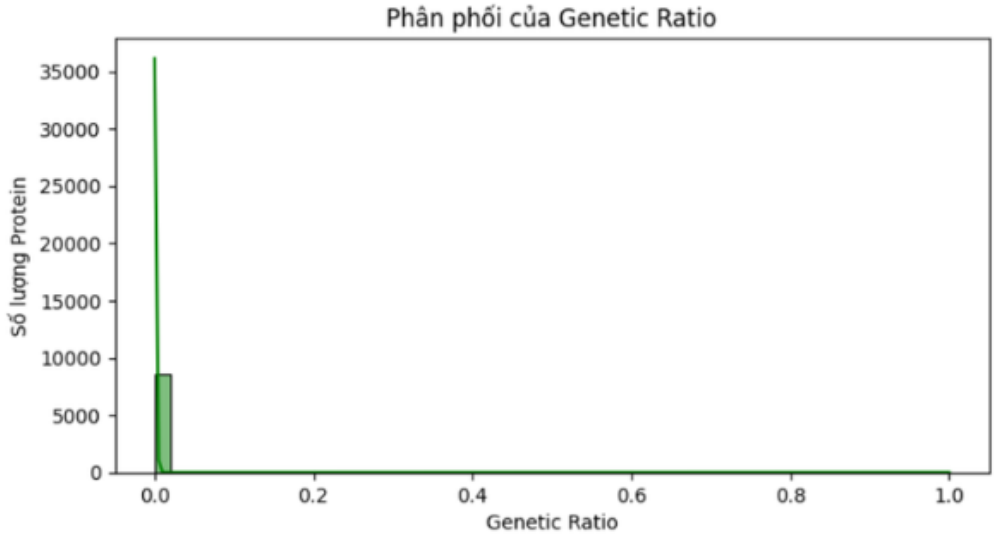
- **Xử lý Dữ liệu**: Sử dụng các cột như *BioGRID ID Interactor A, BioGRID ID Interactor B* để xác định các protein tham gia vào tương tác. Dữ liệu từ cột *Experimental System Type* được dùng để phân loại tương tác thành hai loại: "physical" và "genetic".
- **Đếm Số Lượng Tương Tác**: Đối với mỗi protein, đếm số lần nó tham gia vào các tương tác "physical" và "genetic". Cụ thể:
  - **Tương tác "physical"**: Là các tương tác mà trong đó protein tham gia vào các quá trình vật lý, như hình thành phức hợp protein.
  - **Tương tác "genetic"**: Là các tương tác liên quan đến việc tương tác về mặt di truyền, như sự ảnh hưởng đến gene.
- **Tính Tỷ Lệ**:

- physical_ratio = $\frac{\text{physical_}count}{total}$



Hình 6 Phân bổ của physical ratio

- genetic_ratio = $\frac{\text{genetic_}count}{total}$



Hình 7 Phân bổ của genetic ratio

- **Xử lý Các Giá Trị Thiếu**: Nếu một protein không có bất kỳ tương tác nào (số lượng bằng 0) thì gán tỷ lệ mặc định là 1 để tránh tình trạng chia cho 0.

1.3. Mô hình bài toán

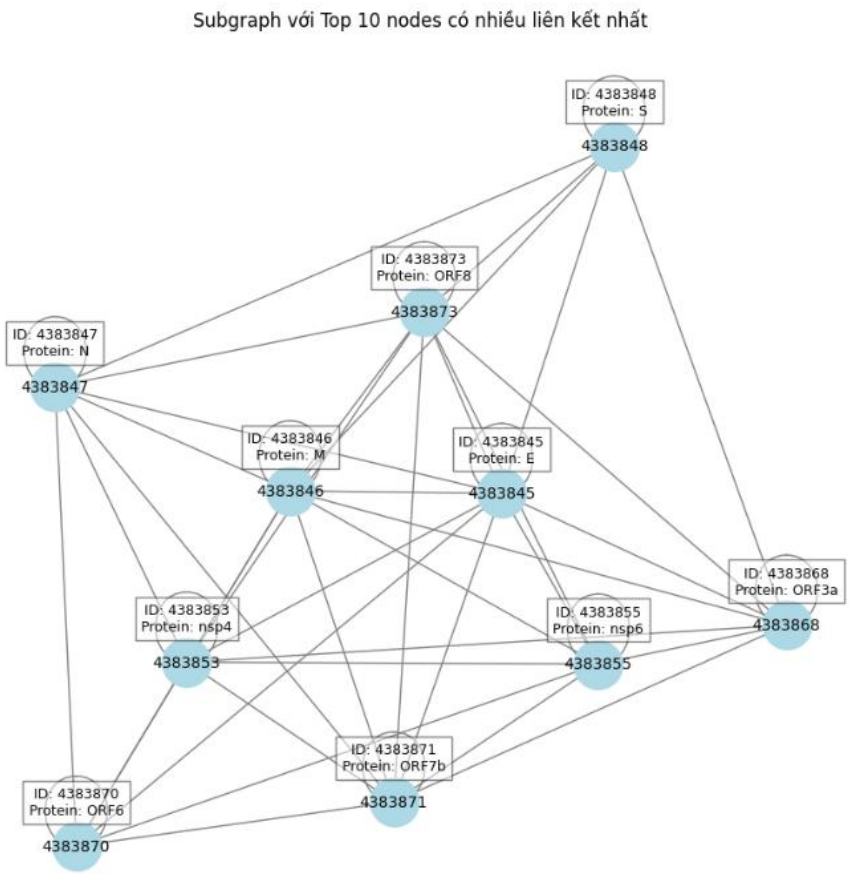
1.3.1. Nodes

- Mỗi protein là một "nút" (node) trên đồ thị.
- Trong bộ dữ liệu, ta tiến hành thu thập các protein từ cột "BioGRID ID Interactor A" và "Official Symbol Interactor A" – đây là hai cột chứa thông tin mã ID và tên của các protein tham gia vào mối liên kết. Tương tự ta thu thập với hai cột "BioGRID ID Interactor B" và "Official Symbol Interactor B" – đây là hai cột chứa thông tin các protein còn lại tham gia vào mối liên kết với các protein ở cột A.
- Sau khi có được danh sách toàn bộ protein, tiến hành loại bỏ các protein trùng nhau để thu được một danh sách chuẩn các protein từ bộ dữ liệu.
- Mỗi nút trên đồ thị có một mã định danh duy nhất (BioGRID ID Interactor – viết tắt là ID) và một tên (Official Symbol Interactor – viết tắt là Protein) đại diện cho mỗi protein.

1.3.2. Edges

- Mỗi tương tác giữa hai protein là một "cạnh" (edge) kết nối hai nút.
- Nếu hai protein có mối quan hệ tương tác với nhau trong bộ dữ liệu, một cạnh kết nối giữa hai nút đại diện cho hai protein đó trên đồ thị sẽ được tạo ra.

1.3.3. Mô hình hóa đồ thị tương tác



Hình 8 Mô hình hóa đồ thị tương tác của 10 nodes có nhiều liên kết nhất

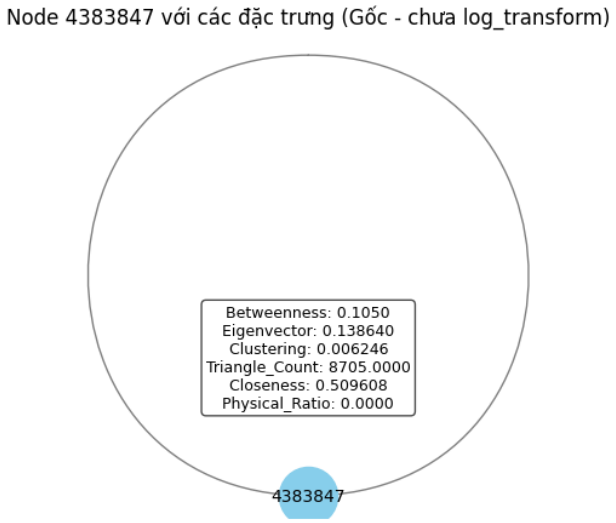
Biểu đồ này thể hiện mạng con (subgraph) được trích xuất từ toàn bộ mạng lưới tương tác protein, bao gồm Top 10 protein có số lượng liên kết nhiều nhất.

Mỗi đỉnh đại diện cho một protein trong virus SARS-CoV-2, có nhãn là ID và tên protein (ví dụ: Protein: S, M, N…).
Mỗi cạnh biểu diễn một tương tác giữa hai protein.

1.3.4. Ma trận đặc trưng đầu vào

Mỗi đỉnh trong đồ thị được biểu diễn bằng một vector đặc trưng, tạo thành ma trận đặc trưng đầu vào. Các đặc trưng được trích xuất gồm:

- Đặc trưng cấu trúc đồ thị:
  - Hệ số gom cụm (clustering coefficient)
  - Độ trung gian (betweenness centrality, log-transform)
  - Vector riêng (eigenvector centrality, log-transform)
  - Số tam giác mà node tham gia (triangle count)
  - Độ gần trung tâm (closeness centrality)
- Đặc trưng sinh học từ dữ liệu BioGRID:
  - Tỷ lệ tương tác vật lý (physical interaction ratio)
  - Tỷ lệ tương tác di truyền (genetic interaction ratio)



Hình 9 Minh họa thông tin của 1 node cụ thể với ID: 4383847 với các đặc trưng ban đầu

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Giới thiệu tổng quan

Graph Convolutional Networks (GCN) là một kiến trúc mạng nơ-ron nhân tạo sâu được thiết kế để hoạt động trên dữ liệu có cấu trúc đồ thị. GCN mở rộng các mạng nơ-ron thông thường bằng cách sử dụng phép toán tích chập trên đồ thị, từ đó cho phép khai thác được mối quan hệ giữa các nút trong đồ thị.

2.2. Nguyên lý hoạt động

GCN hoạt động bằng cách cập nhật biểu diễn của các nút thông qua việc tổng hợp thông tin từ các nút lân cận. Cụ thể, biểu diễn của các nút trong mỗi tầng được cập nhật theo quy tắc lan truyền sau:

$$H^{k+1} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\tilde{A}\widetilde{D}^{-\frac{1}{2}}H^{k}W^{k}\right) \qquad (1)$$

Trong đó:

- $\tilde{A} = A + \mathbf{I}$ là ma trận kề của đồ thị vô hướng G với các kết nối tự thân (self-connections), cho phép tích hợp thông tin đặc trưng của chính nút khi cập nhật biểu diễn của nó.
- $\mathbf{I} \in \mathbb{R}^{N x N}$ là ma trận đơn vị.
- $\widetilde{D}$ là ma trận đường chéo với phần tử đường chéo được xác định bởi $\widetilde{D}_{ii} = \sum_j \tilde{A}_{ij}$.
- σ(·) là hàm kích hoạt như ReLU hoặc Tanh. Hàm kích hoạt ReLU được sử dụng phổ biến và được định nghĩa là: ReLU(x) = max(0, x).
- $W^k \in \mathbb{R}^{F x F'}$ là ma trận biến đổi tuyến tính theo từng lớp, trong đó F và F' lần lượt là kích thước biểu diễn của nút tại tầng thứ k và tầng thứ (k+1). Ma trận này sẽ được tối ưu hóa trong quá trình huấn luyện.

Đối với một nút i, phương trình cập nhật nút có thể được viết lại như sau:

$$H_i^k = \sigma\left(\sum_{j\in\{N(i)Ui\}} \frac{\tilde{A}_{ij}}{\sqrt{\widetilde{D}_{ii}\widetilde{D}_{jj}}} H_j^{k-1}W^k\right) \qquad (2)$$

$$H_i^k = \sigma\left(\sum_{j\in N(i)} \frac{A_{ij}}{\sqrt{\widetilde{D}_{ii}\widetilde{D}_{jj}}} H_j^{k-1}W^k + \frac{1}{\widetilde{D}_i} H_i^{k-1}W^k\right) \qquad (3)$$

Trong phương trình (3), ta có:

- Hàm **AGGREGATE** được định nghĩa là trung bình có trọng số của các biểu diễn nút lân cận. Trọng số của nút lân cận *j* được xác định bởi trọng số của cạnh giữa *i* và *j* (tức là phần tử A_{ij} của ma trận kề), được chuẩn hóa bởi bậc của hai nút.
- Hàm **COMBINE** được định nghĩa là tổng của các thông điệp đã được tổng hợp (**AGGREGATE**) và biểu diễn của chính nút đó, trong đó biểu diễn của nút được chuẩn hóa bởi bậc của chính nó.

Kết nối với Tích chập Phổ trên Đồ thị

Tích chập phổ trên đồ thị có thể được định nghĩa là phép nhân giữa một tín hiệu trên nút $\mathbf{x} \in \mathbb{R}^N$ với một bộ lọc tích chập $g_\theta = \mathrm{diag}(\theta)$ (trong đó $\theta \in \mathbb{R}^N$ là tham số của bộ lọc) trong miền Fourier. Về mặt toán học, điều này được biểu diễn như sau:

$$g_\theta \star \boldsymbol{x} = U g_\theta U^T \boldsymbol{x}. \qquad (4)$$

U đại diện cho ma trận eigenvectors của ma trận Laplacian đồ thị chuẩn hóa $L = I_N - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$. $L = U \Lambda U^T$, trong đó Λ là một ma trận đường chéo các giá trị eigenvalues, $U^T\mathbf{x}$ là phép biến đổi Fourier trên đồ thị của tín hiệu đầu vào **x**. Trong thực tế, g_θ có thể được hiểu là một hàm của các giá trị eigenvalues của ma trận Laplacian đồ thị chuẩn hóa L ($g_\theta(\Lambda)$). Tuy nhiên, việc tính toán trực tiếp phương trình này là rất tốn kém , với độ phức tạp bậc hai theo số lượng nút *N*. Theo nghiên cứu của Hammond et al. (2011), vấn đề này có thể được giải quyết bằng cách xấp xỉ hàm $g_\theta(\Lambda)$ bằng một khai triển đa thức Chebyshev bị cắt ngắn đến bậc *K*:

$$g_{\theta'}(\Lambda) = \sum_{k=0}^{K} \theta'_k T_k(\tilde{\Lambda}), \qquad (5)$$

trong đó:

- $\tilde{\Lambda} = \frac{2}{\lambda_{max}} \Lambda - \mathbf{I}$, với λ_{max} là giá trị eigenvalue lớn nhất của L.
- $\theta' \in \mathbb{R}^K$ là vector của Chebyshev coefficients.
- $T_k(x)$ là những đa thức Chebyshev được định nghĩa đệ quy như sau: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$, với $T_0(x) = 1$ và $T_1(x) = x$.

Bằng cách kết hợp phương trình (4) và phương trình (5), phép tích chập của một tín hiệu *x* với bộ lọc $g_{\theta'}$ có thể được viết lại như sau:

$$g_{\theta'} \star \boldsymbol{x} = \sum_{k=0}^{K} \theta'_k T_k(\tilde{L}) \boldsymbol{x}, \qquad (6)$$

trong đó: $\tilde{L} = \frac{2}{\lambda_{max}} L - \mathbf{I}$.

Từ phương trình này, ta có thể thấy rằng mỗi nút chỉ phụ thuộc vào thông tin trong phạm vi lân cận bậc K. Tổng độ phức tạp của việc tính toán phương trình (6) là O(|E|) (tuyến tính theo số cạnh trong đồ thị gốc G), giúp cho mô hình trở nên rất hiệu quả.

Để xây dựng một mạng nơ-ron dựa trên tích chập trên đồ thị, ta có thể xếp chồng nhiều tầng tích chập được định nghĩa dựa trên phương trình (6), trong đó mỗi tầng được theo sau bởi một phép biến đổi phi tuyến tính. Ở mỗi tầng, thay vì giới hạn mô hình bằng tham số hóa rõ ràng theo đa thức Chebyshev như trong phương trình (6), các tác giả của GCN đề xuất giới hạn số lần tích chập ở $K = 1$ tại mỗi tầng. Bằng cách này, mỗi lớp chỉ định nghĩa một hàm tuyến tính trên ma trận Laplacian của đồ thị L. Tuy nhiên, bằng cách xếp chồng nhiều lớp như vậy, ta vẫn có thể biểu diễn một lớp đa dạng các hàm bộ lọc tích chập trên đồ thị. Về mặt trực quan, mô hình này có khả năng giảm thiểu vấn đề quá khớp (overfitting) với cấu trúc lân cận cục bộ, đặc biệt đối với các đồ thị có phân bố bậc nút có phương sai cao, chẳng hạn như mạng xã hội, World Wide Web và mạng trích dẫn khoa học.

Ở mỗi tầng, ta có thể xấp xỉ thêm rằng $\lambda_{max} \approx 2$, điều này có thể được điều chỉnh thông qua các tham số của mạng nơ-ron trong quá trình huấn luyện. Dựa trên tất cả các giản lược này, ta có:

$$g_{\theta'} \star \boldsymbol{x} \approx \theta'_0 \boldsymbol{x} + \theta'_1 \boldsymbol{x}(L - I_N)\boldsymbol{x} = \theta'_0 \boldsymbol{x} - \theta'_1 D^{-\frac{1}{2}} A D^{-\frac{1}{2}}, \qquad (7)$$

trong đó θ'_0 và θ'_1 là hai tham số tự do, có thể được chia sẻ trên toàn bộ đồ thị. Trong thực tế, ta có thể tiếp tục giảm số lượng tham số, giúp giảm thiểu hiện tượng quá khớp đồng thời tối ưu hóa số phép toán tại mỗi lớp. Kết quả là, ta có thể thu được biểu thức sau:

$$g_\theta \star \boldsymbol{x} \approx \theta \left(\boldsymbol{I} + D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \right) \boldsymbol{x}, \quad (8)$$

trong đó $\theta = \theta'_0 = -\theta'_1$.

Một vấn đề tiềm ẩn là ma trận $I_N + D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$, với các giá trị eigenvalues nằm trong khoảng [0, 2]. Trong một mạng nơ-ron tích chập trên đồ thị sâu, việc lặp đi lặp lại hàm trên có thể dẫn đến hiện tượng gradient bùng nổ hoặc gradient biến mất, gây ra bất ổn số học. Do đó, ta có thể tái chuẩn hóa lại ma trận này bằng cách chuyển đổi $\mathbf{I} + D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$ thành $\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$, trong đó $\tilde{A} = A + \mathbf{I}$ và $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$.

Ở trên, chúng ta chỉ xem xét trường hợp chỉ có một kênh đặc trưng và một bộ lọc. Điều này có thể được mở rộng một cách dễ dàng cho một tín hiệu đầu vào có C kênh đặc trưng $X \in \mathbb{R}^{N x C}$ và F bộ lọc (hoặc số lượng đơn vị ẩn) như sau:

$$H = \widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}XW \qquad (9)$$

trong đó:

$W \in R^{C x F}$ là ma trận chứa các tham số của bộ lọc.

H là ma trận tín hiệu sau khi được tích chập.

2.3. Ứng dụng của graph trong phân tích mạng xã hội

- **Phân tích mạng xã hội:** GCNs có thể được sử dụng để phân tích và dự đoán các quan hệ hoặc hành vi của người dùng trong mạng xã hội.
  Ví dụ: đề xuất bạn bè, phân loại người dùng theo nhóm (community detection), hoặc dự đoán các liên kết tiềm năng giữa người dùng.
- **Đề xuất hệ thống:** Trong các hệ thống gợi ý (recommendation systems), GCNs có thể giúp cải thiện chất lượng gợi ý bằng cách phân tích các mối quan hệ giữa người dùng và sản phẩm, hoặc giữa các sản phẩm với nhau, dựa trên thông tin đồ thị.
- **Học máy trên dữ liệu đồ thị:** GCNs có thể được áp dụng để giải quyết các bài toán phân loại, dự đoán giá trị hoặc phân nhóm trên dữ liệu có dạng đồ thị, như trong bài toán phân loại tài liệu dựa trên các liên kết giữa chúng.
- **Phân tích mạng lưới protein (bioinformatics):** Trong sinh học, các GCNs có thể được sử dụng để dự đoán các mối quan hệ giữa các protein hoặc các mối quan hệ cấu trúc trong mạng lưới các phân tử sinh học.
- **Nhận dạng đối tượng trong hình ảnh:** GCNs có thể được sử dụng trong các tác vụ nhận dạng đối tượng hoặc phân đoạn hình ảnh, đặc biệt khi các đối tượng trong hình ảnh có mối quan hệ không gian hoặc cấu trúc.
- **Phân tích chuỗi cung ứng hoặc mạng lưới logistics:** Các ứng dụng trong kinh tế và logistics cũng có thể sử dụng GCNs để tối ưu hóa các mạng lưới phân phối hoặc dự đoán các sự kiện trong chuỗi cung ứng.
- **Mạng lưới giao thông:** GCNs có thể được áp dụng trong việc dự báo lưu lượng giao thông hoặc phân tích cấu trúc giao thông, nơi các nút trong đồ thị là các giao lộ và các cạnh là các con đường kết nối chúng.
- **Xử lý ngữ nghĩa trong NLP (Xử lý ngôn ngữ tự nhiên):** Các GCNs có thể được áp dụng để học các quan hệ ngữ nghĩa giữa các từ hoặc các thực thể trong văn bản, cải thiện khả năng hiểu văn bản và thực hiện các tác vụ như dịch máy, phân loại văn bản, và trích xuất thông tin.

2.4. So sánh GCNs và GATs

2.4.1. Đặc trưng

Bảng 8 So sánh đặc trưng GCNs và GATs

Đặc trưng/Graph	GCNs	GATs
Cơ chế hoạt động	GCNs sử dụng phép toán **convolution (chập)** để cập nhật các đặc trưng của nút. Mỗi nút trong đồ thị sẽ nhận thông tin từ các nút lân cận và tính toán đặc trưng mới bằng cách lấy trung bình có trọng số các đặc trưng từ các nút lân cận.	GATs sử dụng cơ chế **attention (chú ý)** để học trọng số riêng biệt cho từng cạnh trong đồ thị, thay vì sử dụng trọng số đồng nhất cho tất cả các liên kết. Mỗi nút tính toán sự chú ý đối với các nút lân cận, và trọng số của mỗi cạnh được học từ dữ liệu.
Cấu trúc	Mỗi lớp trong GCN kết hợp các đặc trưng của các nút lân cận và bản thân nó để cập nhật thông tin. Các **trọng số giữa các nút thường là đồng nhất**, tức là tất cả các cạnh giữa các nút đều có trọng số giống nhau.	Mỗi lớp GAT có thể học các **trọng số riêng biệt** cho từng liên kết, giúp mô hình có khả năng đánh giá mức độ quan trọng của mỗi liên kết trong việc cập nhật đặc trưng của các nút.
Khả năng điều chỉnh trọng số cạnh	Trọng số giữa các nút là đồng nhất và không thay đổi tùy thuộc vào các nút. Tất cả các cạnh trong đồ thị đều có trọng số giống nhau, điều này có thể gây **khó khăn khi các mối quan hệ giữa các nút có sự khác biệt lớn về mức độ quan trọng.**	Sử dụng cơ chế attention để tự động học trọng số riêng cho từng cạnh, điều này giúp mô hình **có thể phân biệt các nút lân cận quan trọng và không quan trọng**. Trọng số của các cạnh là không đồng nhất và thay đổi theo mối quan hệ giữa các nút.
Khả năng xử lý đồ thị không đồng nhất	GCNs có thể gặp khó khăn khi xử lý các đồ thị không đồng nhất (các nút có mối quan hệ khác nhau hoặc độ	GATs rất hiệu quả trong việc xử lý các đồ thị không đồng nhất. Nhờ vào cơ chế attention, GATs **có thể**

	quan trọng không đồng đều). Mô hình này **không có khả năng tự động học tầm quan trọng của các nút trong đồ thị**.	**học tầm quan trọng của từng nút và điều chỉnh trọng số cho các cạnh một cách linh hoạt.**
Tính linh hoạt	GCNs có tính đơn giản và dễ hiểu. Tuy nhiên, chúng **ít linh hoạt hơn** so với GATs trong việc học các mối quan hệ phức tạp giữa các nút. Đặc biệt, GCN không thể phân biệt rõ ràng tầm quan trọng của các nút trong các đồ thị phức tạp.	GATs **có tính linh hoạt cao** hơn nhờ cơ chế attention, cho phép mô hình tự động điều chỉnh trọng số cho các cạnh và học các mối quan hệ phức tạp giữa các nút. Điều này làm cho GATs đặc biệt hiệu quả trong các ứng dụng đòi hỏi sự chú ý đến các mối quan hệ không đồng nhất.
Khả năng mở rộng	GCNs có thể gặp **khó khăn khi mở rộng với các đồ thị lớn và dày đặc** vì phép toán chập cần tính toán cho tất cả các nút và các nút lân cận của chúng. Điều này có thể gây tốn kém về thời gian và bộ nhớ.	GATs cũng **có thể gặp vấn đề về hiệu suất khi mở rộng với các đồ thị rất lớn, nhưng cơ chế attention có thể giúp cải thiện tính linh hoạt** trong việc xử lý các cạnh quan trọng và giảm thiểu số lượng tính toán không cần thiết.
Tài nguyên tính toán và hiệu suất	Hiệu suất tính toán của GCNs thường **nhanh hơn** so với GATs vì không có cơ chế attention phức tạp. Tuy nhiên, GCNs có thể không khai thác tối đa thông tin trong các đồ thị không đồng nhất, dẫn đến **hiệu suất kém hơn trong các bài toán phức tạp.**	**Tài nguyên tính toán của GATs thường cao** hơn vì phải tính toán attention cho từng cạnh. Tuy nhiên, sự linh hoạt trong việc học trọng số cạnh riêng biệt có thể giúp GATs đạt được **hiệu suất tốt hơn trong các bài toán phức tạp**

		hoặc khi dữ liệu có tính không đồng nhất cao.
Ứng dụng	GCNs phù hợp với các bài toán có **đồ thị đồng nhất,** chẳng hạn như phân loại nút trong mạng xã hội hoặc **đồ thị có cấu trúc đơn giản.**	GATs đặc biệt phù hợp cho các bài toán có **đồ thị không đồng nhất hoặc yêu cầu mô hình phải học mối quan hệ phức tạp giữa các nút**, chẳng hạn như phân tích mạng xã hội, nhận dạng đối tượng trong đồ thị, hoặc các ứng dụng sinh học.

2.4.2. Điểm mạnh

Bảng 9 So sánh điểm mạnh GCNs và GATs

GCNs	**GATs**
• Hiệu suất tốt trên đồ thị đồng nhất: GCNs hoạt động rất tốt trên các đồ thị mà các quan hệ giữa các nút có tính đồng nhất, như trong mạng xã hội hoặc mạng con người. • Đơn giản và dễ hiểu: Cấu trúc của GCN khá đơn giản và dễ hiểu, với quá trình truyền thông tin giữa các nút dựa trên một phép toán convolution. • Tiết kiệm tài nguyên tính toán: Vì GCN sử dụng trọng số cố định cho tất cả các liên kết, việc tính toán thường đơn giản và ít yêu cầu tài nguyên tính toán hơn so với các mô hình phức tạp hơn.	• Khả năng học trọng số linh hoạt: GATs sử dụng cơ chế attention để học trọng số cạnh linh hoạt, giúp mô hình điều chỉnh mức độ quan trọng của từng nút lân cận, từ đó cải thiện hiệu suất trên đồ thị không đồng nhất. • Khả năng xử lý đồ thị không đồng nhất: Vì trọng số của các cạnh có thể thay đổi theo mối quan hệ, GATs có thể dễ dàng xử lý các đồ thị phức tạp, nơi các mối quan hệ giữa các nút có tính chất không đồng nhất. • Hiệu quả trong bài toán phức tạp: GATs đặc biệt hiệu quả trong các bài toán mà quan hệ giữa các nút rất phức tạp hoặc thay đổi, như phân tích mạng xã hội, hệ thống đề xuất, hoặc sinh học.

2.4.3. Điểm yếu

Bảng 10 So sánh điểm yếu của GCNs và GATs

GCNs	GATs
• Khó xử lý đồ thị không đồng nhất: GCNs không thể học các trọng số cạnh linh hoạt, điều này hạn chế khả năng xử lý các đồ thị mà các mối quan hệ giữa các nút không đồng nhất. • Khó mở rộng với đồ thị lớn: Khi số lượng nút và cạnh trong đồ thị rất lớn, GCNs có thể gặp vấn đề về tính toán và hiệu suất, đặc biệt trong các đồ thị dày đặc. • Thông tin từ các nút lân cận: Các GCNs đơn giản không thể phân biệt giữa các nút lân cận quan trọng và không quan trọng, tất cả đều được xử lý với trọng số giống nhau.	• Yêu cầu tài nguyên tính toán cao: Việc tính toán attention cho mỗi cạnh có thể rất tốn kém, đặc biệt trong các đồ thị lớn. Mỗi bước tính toán attention yêu cầu phải tính toán cho từng cặp nút, điều này có thể làm tăng chi phí tính toán. • Khó kiểm soát: Mặc dù GATs linh hoạt hơn, nhưng điều này cũng có thể dẫn đến việc khó kiểm soát quá trình học và dẫn đến overfitting nếu không điều chỉnh đúng cách. • Chậm hơn so với GCNs: Do cơ chế attention, quá trình huấn luyện và dự đoán có thể chậm hơn so với GCNs, đặc biệt trên các đồ thị lớn và dày đặc.

2.5. Các đặc trưng của graph

Trung tâm đồ thị (Graph Centrality)

Định nghĩa: Một số node có vai trò "trung tâm" (central) trong đồ thị (ví dụ: node có bậc cao, betweenness cao).

Xử lý trong GCN: GCN không trực tiếp xem xét centrality, nhưng cấu trúc đồ thị tự động ưu tiên các node có nhiều kết nối.

Các chỉ số trung tâm đã sử dụng trực tiếp làm đặc trưng đầu vào:

Eigenvector Centrality – Mức độ ảnh hưởng

Ý nghĩa: Một protein quan trọng nếu nó kết nối với các protein cũng quan trọng.

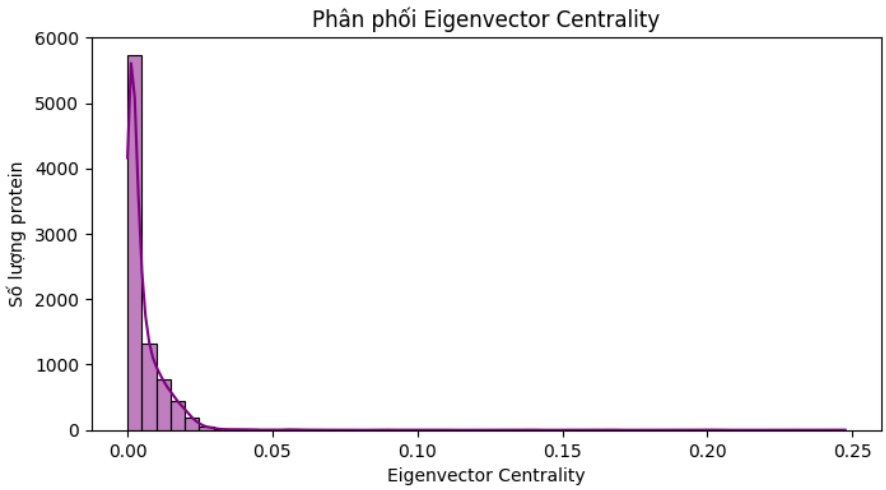
Ứng dụng: Xác định protein có ảnh hưởng sâu rộng, không chỉ đơn thuần có nhiều kết nối.

Eigenvector Centrality được tính bằng:

$$x_i = \frac{1}{\lambda} \sum_{j=1}^{n} A_{ij} x_j$$

Trong đó:

A: Ma trận kề (adjacency matrix) của đồ thị
x_i: Giá trị centrality của node i
λ: Giá trị riêng (eigenvalue) lớn nhất tương ứng
x: Vector đặc trưng chứa centrality của tất cả các node



Hình 10 Phân phối Eigenvector theo số lượng protein

Hình 11 Top 20 protein theo eigenvector

Closeness Centrality – Mức độ gần trung tâm

Ý nghĩa: Đo khoảng cách trung bình từ một protein đến tất cả các protein khác trong mạng.

Ứng dụng: Protein có closeness cao thường truyền tín hiệu hoặc tác động nhanh đến các phần còn lại.

Closeness Centrality được tính bằng:

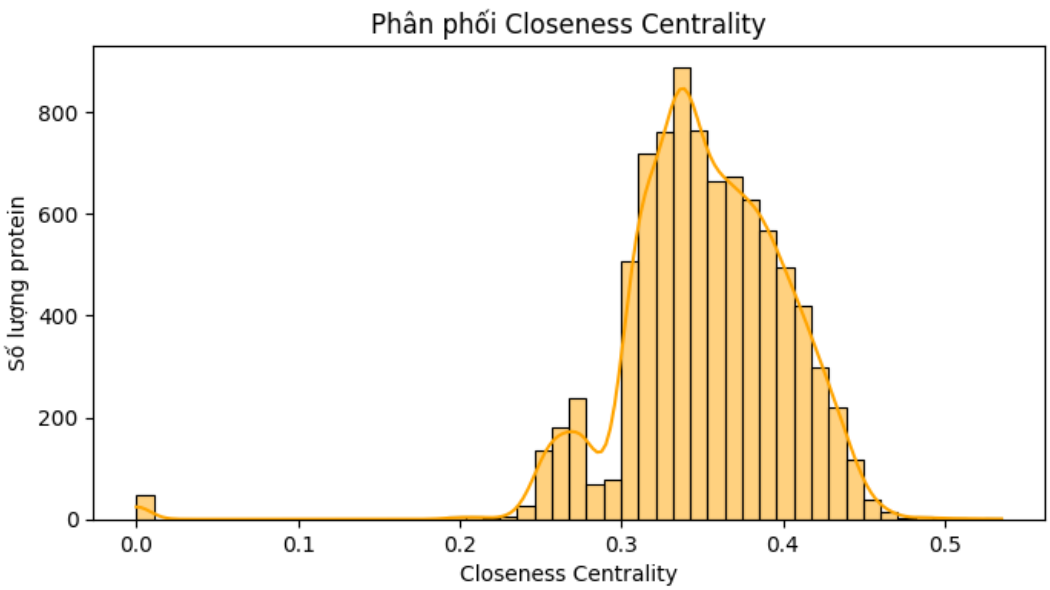
$$C(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)}$$

Trong đó:

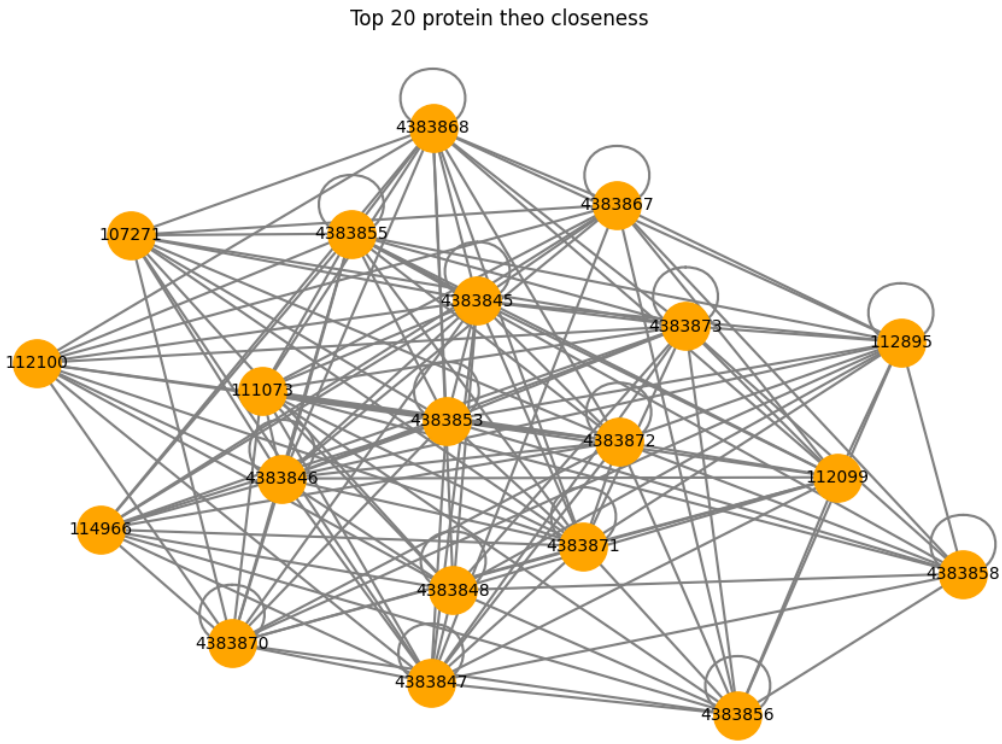
$C(v)$: closeness centrality của đỉnh v

n: tổng số đỉnh trong đồ thị

$d(v, u)$: độ dài đường đi ngắn nhất từ đỉnh v đến đỉnh u



Hình 12 Phân phối protein theo Closeness Centrality



Hình 13 Top 20 protein theo Closeness Centrality

Betweenness Centrality – Mức độ cầu nối

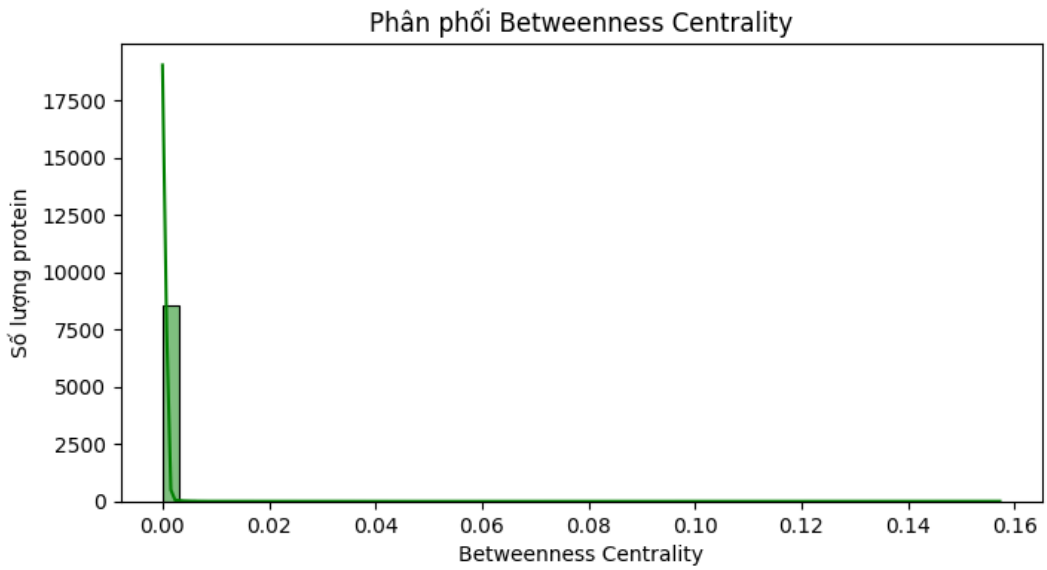
Ý nghĩa: Số lần một protein nằm trên đường đi ngắn nhất giữa các cặp protein khác.

Ứng dụng: Protein này như "cầu nối" quan trọng – ngắt nó có thể chia mạng.
Betweenness Centrality được tính bằng:

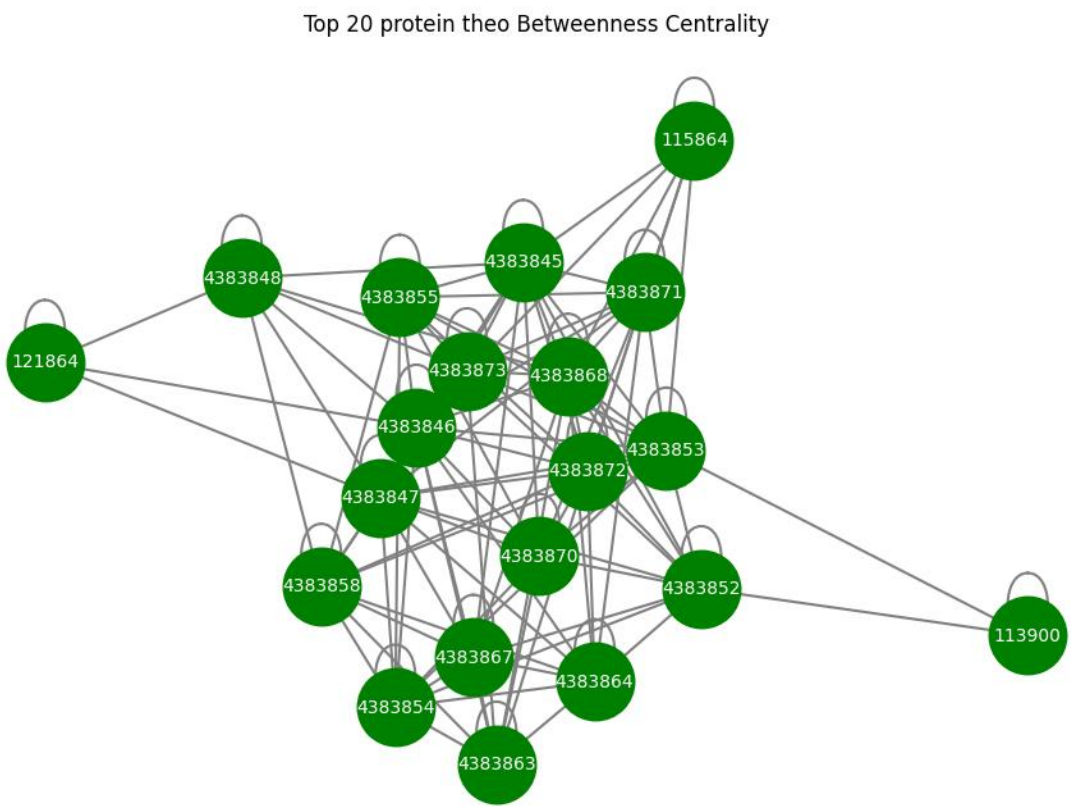
$$C_B(v) = \sum_{s \neq v \neq t} \frac{\boldsymbol{\sigma}_{st}(v)}{\boldsymbol{\sigma}_{st}}$$

Trong đó:

$C_B(v)$: Betweenness centrality của đỉnh *v*
$\boldsymbol{\sigma}_{st}$: Tổng số đường đi ngắn nhất từ đỉnh *s* đến đỉnh *t*
$\boldsymbol{\sigma}_{st}(v)$: Số đường đi ngắn nhất từ *s* đến *t* đi qua *v*
Tổng được tính trên tất cả các cặp *s*, *t* mà $s \neq v \neq t$



Hình 14 Phân phối protein theo Betweeness Centrality



Hình 15 Top 20 protein theo Betweeness Centrality

- Bên cạnh đó, còn sử dụng các đặc trưng graph khác như:

  Clustering Coefficient

  Định nghĩa: Đo lường mức độ các node hàng xóm của một node cũng kết nối với nhau.

  Ý nghĩa trong mạng xã hội: Nếu một node có Clustering Coefficient cao, và một node khác cũng tương tự, khả năng có liên kết giữa chúng sẽ cao hơn.

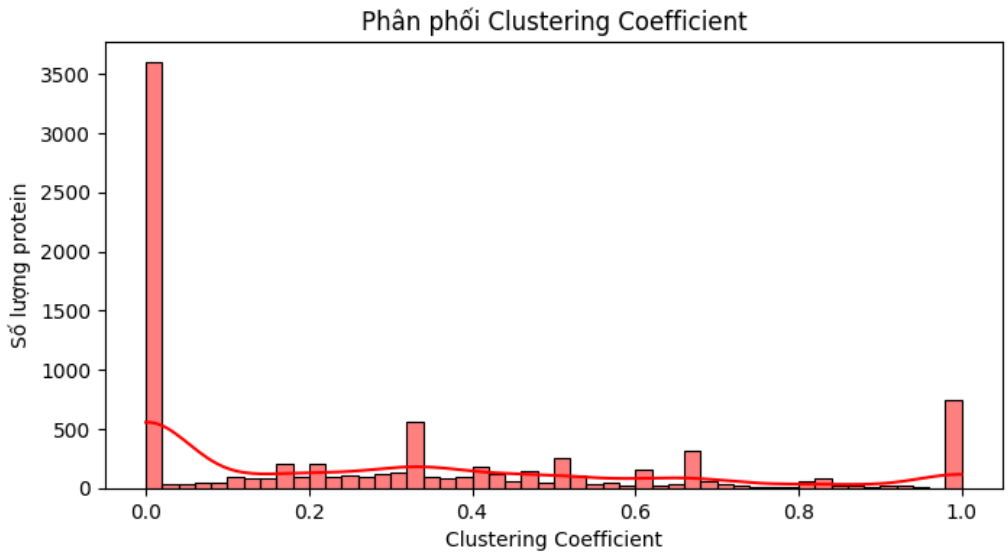
  Clustering Coefficient được tính bằng:

$$C(v) = \frac{2T(v)}{k_v(k_v - 1)}$$

Trong đó:

$T(v)$: Số tam giác mà đỉnh v tham gia (tức là các cặp hàng xóm của v có kết nối với nhau tạo thành tam giác với v).

k_v: Bậc của đỉnh v (tức là số lượng hàng xóm trực tiếp mà v kết nối).



Hình 16 Phân phối protein theo Clustering

Triangle Count (TC)

Định nghĩa: Là số lượng tam giác (3 node liên kết với nhau từng cặp) mà một node tham gia.

Ý nghĩa trong mạng xã hội: Đại diện cho mức độ gắn kết cộng đồng — node có nhiều tam giác thường là trung tâm trong các nhóm, mạng liên kết chặt chẽ.

TC được tính bằng:

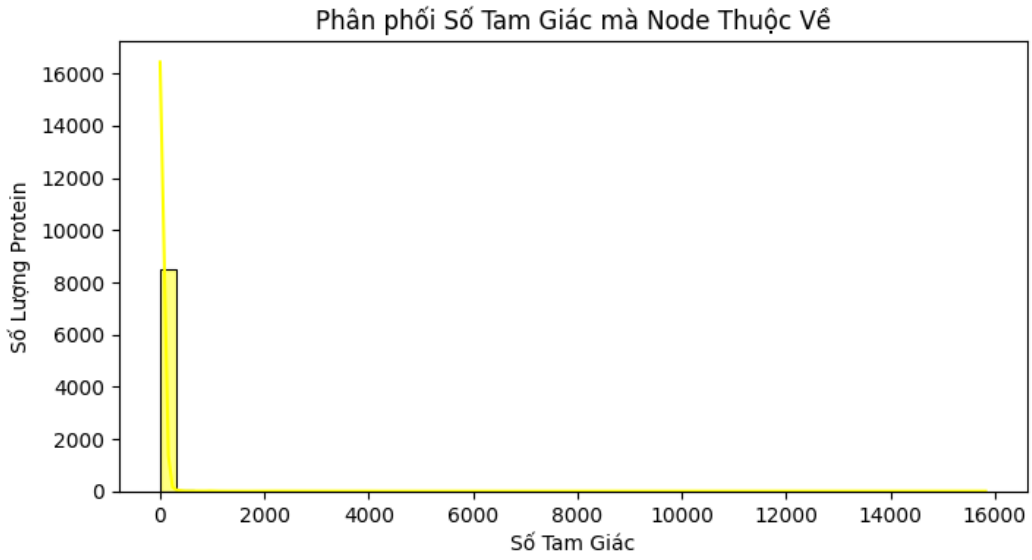
$$TriangleCount(v) = \frac{1}{2} \sum_{u,w \in N(v)} 1_{\{(u,w) \in E\}}$$

Trong đó:

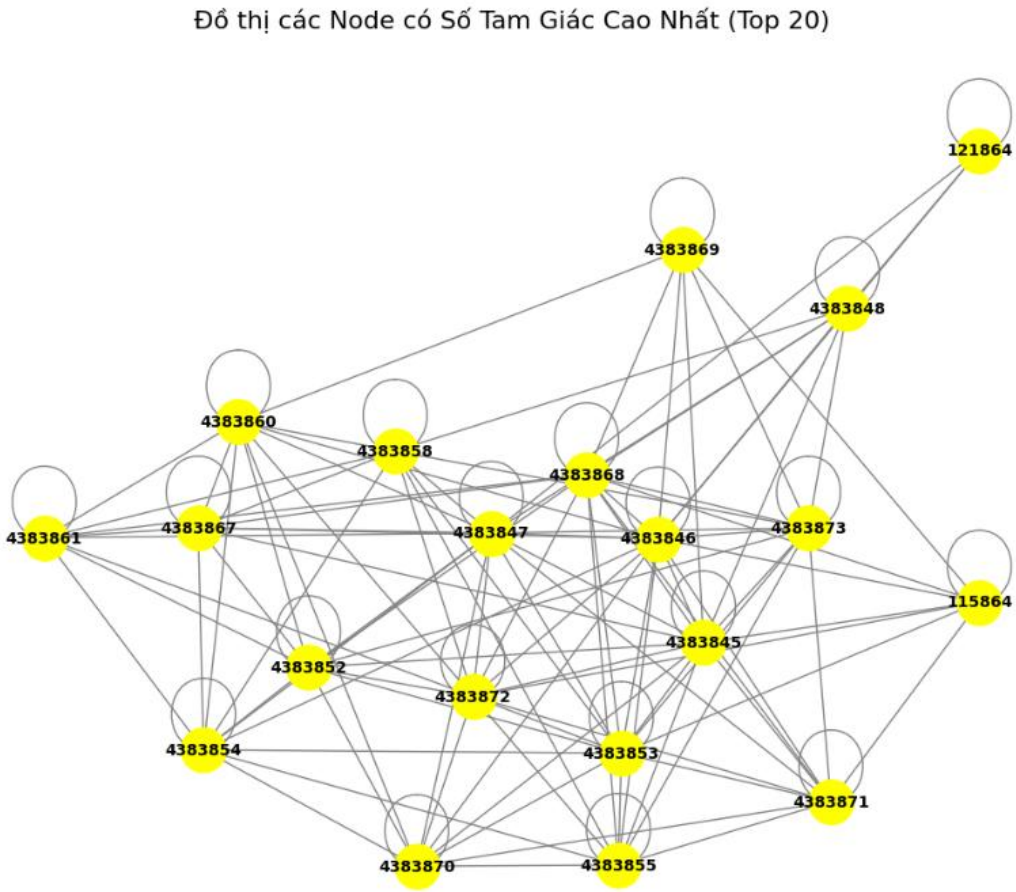
$N(v)$: tập hàng xóm của đỉnh v

$1_{\{(u,w) \in E\}}$: bằng 1 nếu giữa u và w có cạnh, tức là tạo thành tam giác với v, ngược lại là 0.

Hệ số $\frac{1}{2}$: vì mỗi tam giác sẽ bị đếm 2 lần do tổ hợp cặp (u, w)



Hình 17 Phân phối protein theo Triangle Count



Hình 18 Top 20 node có Triangle Count cao nhất

2.6. Bài báo về graph GCN

2.6.1. Giới thiệu bài báo

- **Tên bài báo:** *Integrating Sequence and Network Information to Enhance Protein-Protein Interaction Prediction Using Graph Convolutional Networks* [1]
- **Tác giả:** Leilei Liu, Xianglei Zhu, Yi Ma, Yaodong Yang, Xiaotian Hao, Li Wang, và Jiajie Peng
- **Tạp chí:** Hội nghị quốc tế IEEE về Tin sinh học và Y sinh học (BIBM - Bioinformatics and Biomedicine), năm 2019.

2.6.2. Mục tiêu nghiên cứu

Bài báo hướng đến việc nâng cao độ chính xác trong dự đoán tương tác protein-protein (PPI) bằng cách kết hợp hai nguồn thông tin quan trọng: chuỗi amino acid của protein và cấu trúc mạng lưới tương tác protein. Mục tiêu là xây dựng một mô hình học sâu có khả năng tận dụng đồng thời thông tin nội tại từ trình tự protein và thông tin cấu trúc từ mạng PPI để dự đoán xác suất hai protein tương tác với nhau.

2.6.3. Phương pháp thực hiện

Phương pháp được chia thành hai giai đoạn chính:

- **Giai đoạn biểu diễn đặc trưng:**

  - *Biểu diễn chuỗi protein:* Mỗi amino acid được mã hóa bằng one-hot vector, và chuỗi protein được ánh xạ vào một ma trận 20×20. Để xử lý độ dài khác nhau, chuỗi ngắn được đệm thêm 0, chuỗi dài bị cắt ngắn.
  - *Biểu diễn đồ thị PPI:* Mạng tương tác protein được mô hình hóa dưới dạng đồ thị vô hướng không trọng số, trong đó mỗi đỉnh là một protein, cạnh biểu thị mối tương tác. Đặc trưng ban đầu là ma trận đơn vị n × n. Một tầng GCN theo hướng tiếp cận phổ (spectral GCN) được sử dụng để truyền thông tin giữa các đỉnh thông qua cơ chế lan truyền thông điệp (message passing), theo công thức:

$$X_1 = \mathrm{ReLU}\left(\widetilde{D^{-1}}\,\tilde{A}\,X_0\,W_0\right)$$

    Trong đó:

    - $\tilde{A}$: Ma trận kề có thêm self-loop
    - $\widetilde{D}$: Ma trận bậc tương ứng với $\tilde{A}$
    - W_0 : Ma trận trọng số huấn luyện
    - X_0: Ma trận đặc trưng ban đầu
    - X_1: đặc trưng đầu ra cho mỗi protein
    - ReLU: Hàm kích hoạt ReLU

- **Giai đoạn dự đoán:**

- Hai mạng DNN riêng biệt (gồm các tầng fully connected, batch normalization và dropout) được áp dụng cho hai vector đặc trưng tương ứng với hai protein cần kiểm tra.
- Đầu ra của hai mạng được nối lại và đưa vào một mạng hợp nhất, sử dụng hàm softmax để dự đoán xác suất tương tác giữa hai protein.

2.6.4. Điểm mạnh

- **Tích hợp hiệu quả hai nguồn thông tin là thông tin chuỗi amino acid của protein và cấu trúc mạng lưới protein (PPI).**

  Điều này giúp mô hình có thể khai thác được những đặc trưng bổ sung từ cả hai phía, từ đó tạo ra một đại diện mạnh mẽ và toàn diện hơn về mỗi protein. Thông tin chuỗi cung cấp các đặc trưng cấu trúc bậc một của protein, trong khi thông tin đồ thị giúp khai thác các mối quan hệ tương tác giữa các protein, hỗ trợ việc dự đoán chính xác hơn.
- **Không cần huấn luyện embedding phức tạp, tiết kiệm tài nguyên và thời gian:**

  Không cần sử dụng các kỹ thuật embedding phức tạp như Word2Vec hay PSI-BLAST để mã hóa thông tin chuỗi protein.

  Phương pháp sử dụng one-hot encoding đơn giản, không cần huấn luyện tham số, điều này không chỉ giúp giảm bớt sự phức tạp trong quá trình huấn luyện mà còn tiết kiệm thời gian và tài nguyên tính toán.

  Đồng thời, việc không sử dụng các embedding phức tạp cũng giúp giảm thiểu độ phức tạp của mô hình, làm cho nó dễ triển khai và sử dụng.
- **Kiến trúc mạng đơn giản, ít tham số nhưng hiệu quả cao:**

  Phương pháp đề xuất sử dụng một kiến trúc mạng đơn giản và ít tham số hơn so với các mô hình khác như DPPI hay DeepFE-PPI. Thay vì sử dụng các mô-đun phức tạp như convolutional layers hay học embedding, phương pháp của nhóm tác giả chỉ sử dụng một tầng GCN để xử lý thông tin đồ thị, cùng với hai mạng DNN đơn giản để dự đoán khả năng tương tác. Dù có ít tham số hơn, phương pháp này vẫn đạt được hiệu suất vượt trội, chứng minh rằng một mô hình đơn giản có thể hoạt động hiệu quả trong bài toán này.

Bảng 11 Đầu vào và kiến trúc của các phương pháp

	Đầu vào	**Kiến trúc**
DPPI	Hồ sơ xác suất từ PSI-BLAST	5 lớp CNN → Random Projection → Dự đoán
DeepFE-PPI	Vector từ Word2Vec	4 lớp fully connected: 2048 → 1024 → 512 → 128

Phương pháp của nhóm tác giả	**Chuỗi + đồ thị**	**GCN + DNN: 256 → 128 → 64 → 32**

- **Hiệu suất vượt trội so với các phương pháp khác:**

Các kết quả thực nghiệm chỉ ra rằng phương pháp của nhóm tác giả đạt kết quả tốt hơn so với các mô hình khác như DPPI và DeepFE-PPI trên nhiều tập dữ liệu chuẩn. Cụ thể, độ chính xác (Accuracy), độ nhạy (Recall) và độ chính xác của mô hình (Precision) đều cao hơn, cho thấy rằng việc kết hợp thông tin chuỗi và đồ thị giúp cải thiện đáng kể chất lượng dự đoán.

Bảng 12 So sánh các chỉ số hiệu suất

	Precision	**Recall**	**Accuracy**
DPPI	0.9668	0.9224	0.9455
DeepFE-PPI	0.9645	0.9299	0.9478
Phương pháp của nhóm tác giả	**0.9702**	**0.9355**	**0.9533**

Trong đó:

- Precision đo lường khả năng của mô hình trong việc dự đoán đúng các trường hợp thuộc về lớp dương tính (positive class). Nó cho biết tỷ lệ các dự đoán dương tính mà mô hình đưa ra là chính xác.
- Recall đo lường khả năng của mô hình trong việc phát hiện tất cả các trường hợp thực sự là dương tính trong dữ liệu. Nó cho biết tỷ lệ các trường hợp dương tính thực sự mà mô hình đã nhận diện đúng.
- Accuracy đo lường tỷ lệ các dự đoán đúng (bao gồm cả dương tính và âm tính) so với tổng số trường hợp trong bộ dữ liệu. Đây là một chỉ số tổng quát về hiệu suất của mô hình.

- **Tổng quát tốt trên nhiều tập dữ liệu, bao gồm cả tập dữ liệu mất cân bằng:**

Đối với các tập dữ liệu mất cân bằng (ví dụ: Human và Yeast), tác giả sử dụng thêm đường cong Precision-Recall (PR) và diện tích dưới đường cong (auPR) để đánh giá.

Trong đó:

- Đường cong PR cho thấy sự đánh đổi giữa độ chính xác và độ phủ – đặc biệt hữu ích trong các bài toán có dữ liệu lệch lớp.
- Diện tích lớn hơn dưới đường cong thể hiện mô hình có khả năng cân bằng tốt giữa độ chính xác và độ phủ.

Phương pháp có khả năng tổng quát tốt trên nhiều tập dữ liệu khác nhau, bao gồm cả các tập dữ liệu cân bằng và mất cân bằng. Đây là một vấn đề phổ biến trong các bài toán sinh học, nơi số lượng tương tác giữa các protein có thể không đồng đều. Phương pháp này vẫn duy trì được hiệu suất cao và ổn định ngay cả khi đối mặt với các dữ liệu mất cân bằng, cho thấy tính linh hoạt và khả năng ứng dụng.

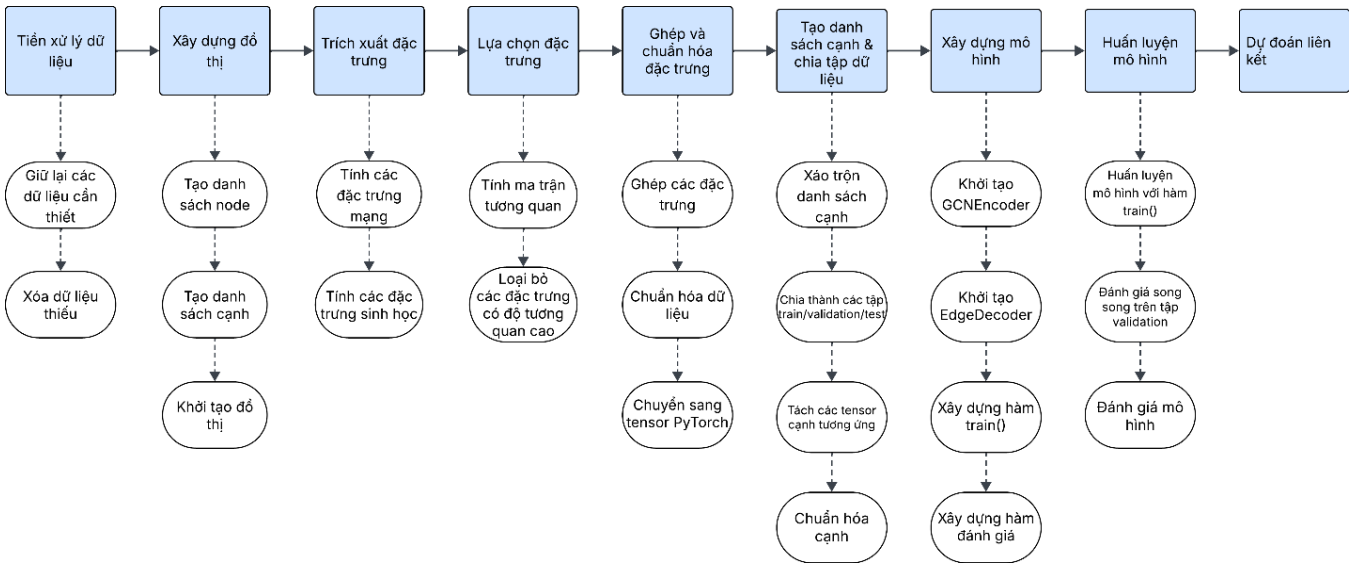
- **Đơn giản và dễ triển khai:**
  Với kiến trúc mạng đơn giản và không yêu cầu huấn luyện các embedding phức tạp, phương pháp này dễ dàng triển khai và sử dụng trong các nghiên cứu và ứng dụng thực tế. Điều này giúp giảm bớt yêu cầu tính toán và tài nguyên, đồng thời giúp các nhà nghiên cứu nhanh chóng áp dụng vào các dự án của mình mà không gặp phải các khó khăn trong việc chuẩn bị dữ liệu hay huấn luyện mô hình.
- **Tính ổn định và khả năng ứng dụng cao:**
  Kết quả trên các tập dữ liệu chuẩn và sự so sánh với các phương pháp khác cho thấy tính ổn định cao của phương pháp này. Mô hình có thể được mở rộng và tùy chỉnh để phù hợp với các bài toán khác nhau, mở ra nhiều cơ hội trong nghiên cứu và ứng dụng thực tế.

2.6.5. Điểm yếu

- **Biểu diễn chuỗi còn đơn giản:** Phương pháp sử dụng one-hot encoding cho chuỗi amino acid.
- **Khả năng tổng quát hóa hạn chế với dữ liệu mới:** Phương pháp sử dụng spectral GCN, một kỹ thuật phụ thuộc vào cấu trúc phổ của đồ thị.
- **Chỉ sử dụng một tầng GCN:** Mặc dù điều này giúp giảm độ phức tạp và chi phí tính toán, nhưng đồng thời giới hạn khả năng truyền và học thông tin từ các đỉnh xa hơn trong đồ thị, có thể bỏ sót các mối quan hệ quan trọng trong mạng tương tác protein.
- **Kiến trúc học sâu còn đơn giản:** Mô hình chỉ sử dụng một số lớp fully connected, normalization và dropout. So với kiến trúc sâu và có cơ chế trích chọn đặc trưng tốt hơn (như convolution trong DPPI hoặc tầng học phức tạp trong DeepFE-PPI), mô hình này có thể chưa học được các đặc trưng phi tuyến tính phức tạp trong dữ liệu.

CHƯƠNG 3: HIỆN THỰC

3.1. Sơ đồ hóa các bước chạy chương trình



Hình 19 Sơ đồ hoá các bước chạy chương trình

3.2. Giải thích các bước chạy chương trình

3.2.1. Tiền xử lý dữ liệu

Giữ lại một số cột cần thiết trong bộ dữ liệu để phục vụ cho việc xây dựng mô hình. Các cột được giữ lại: BioGRID ID Interactor A, Official Symbol Interactor A, BioGRID ID Interactor B, Official Symbol Interactor B, Experimental System Type và Experimental System.

Sau đó, tiến hành xóa các hàng dữ liệu có giá trị rỗng.

```
print(df.head())

   BioGRID ID Interactor A Official Symbol Interactor A  \
0                    31542                         MCM5
1                    32899                         MCM7
2                    31542                         MCM5
3                    36697                         MCM3
4                    32899                         MCM7

   BioGRID ID Interactor B Official Symbol Interactor B  \
0                    32899                         MCM7
1                    31542                         MCM5
2                    36697                         MCM3
3                    31542                         MCM5
4                    31376                        CDC45

  Experimental System Type Experimental System
0                 physical          Two-hybrid
1                 physical          Two-hybrid
2                 physical          Two-hybrid
3                 physical          Two-hybrid
4                 physical          Two-hybrid
```

Hình 20 Năm dòng đầu của dữ liệu đã được tiền xử lý

3.2.2. Xây dựng đồ thị

Chuẩn bị dữ liệu đồ thị từ bảng tương tác protein và xây dựng một đồ thị vô hướng bằng thư viện NetworkX, trong đó:

Đỉnh (node): các protein (hoặc gen).
Cạnh (edge): mối tương tác giữa hai protein.

Trong bộ dữ liệu, tiến hành thu thập các protein từ cột "BioGRID ID Interactor A" và "Official Symbol Interactor A" – đây là hai cột chứa thông tin mã ID và tên của các protein tham gia vào mối liên kết. Tương tự, thu thập với hai cột "BioGRID ID Interactor B" và "Official Symbol Interactor B" – đây là hai cột chứa thông tin các protein còn lại tham gia vào mối liên kết với các protein ở cột A.

Sau khi có được danh sách toàn bộ protein, tiến hành loại bỏ các protein trùng nhau để thu được một danh sách chuẩn các protein từ bộ dữ liệu.

Mỗi nút trên đồ thị có một mã định danh duy nhất (BioGRID ID Interactor – viết tắt là ID) và một tên (Official Symbol Interactor – viết tắt là Protein) đại diện cho mỗi protein.

Mỗi tương tác giữa hai protein là một "cạnh" (edge) kết nối hai nút. Nếu hai protein có mối quan hệ tương tác với nhau trong bộ dữ liệu, một cạnh kết nối giữa hai nút đại diện cho hai protein đó trên đồ thị sẽ được tạo ra.

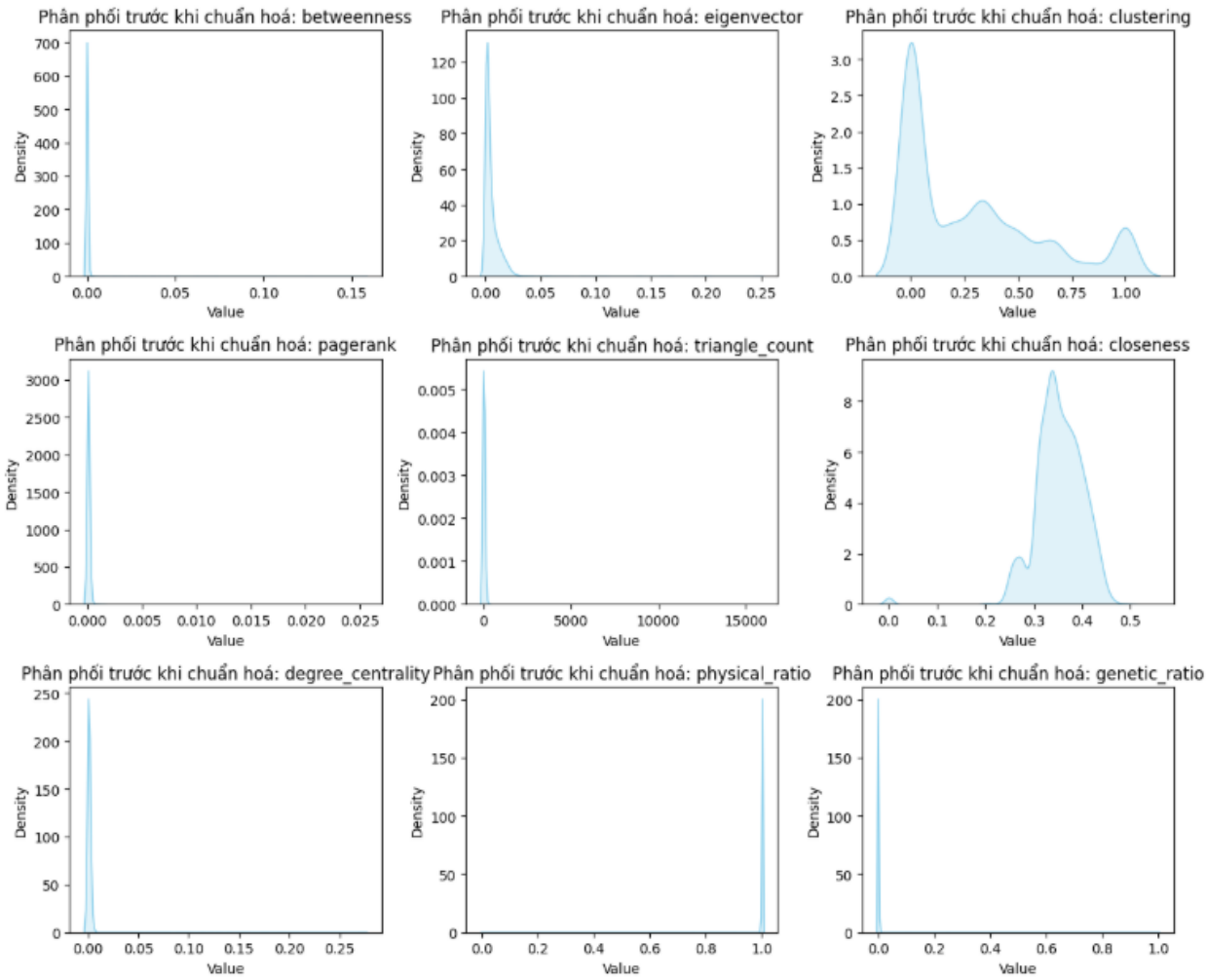
3.2.3. Trích xuất đặc trưng

Tính toán các đặc trưng từ đồ thị sau:

- Clustering
- Pagerank
- Betweenness
- Eigenvector
- Triangle_count
- Closeness centrality
- Degree centrality

Xây dựng một hàm để trích xuất các đặc trưng sinh học, Physical_ratio và Genetic_ratio, từ bộ dữ liệu.

Sau khi đã trích xuất tất cả các đặc trưng, tạo một dataframe để chứa các đặc trưng ấy của các node trong đồ thị.



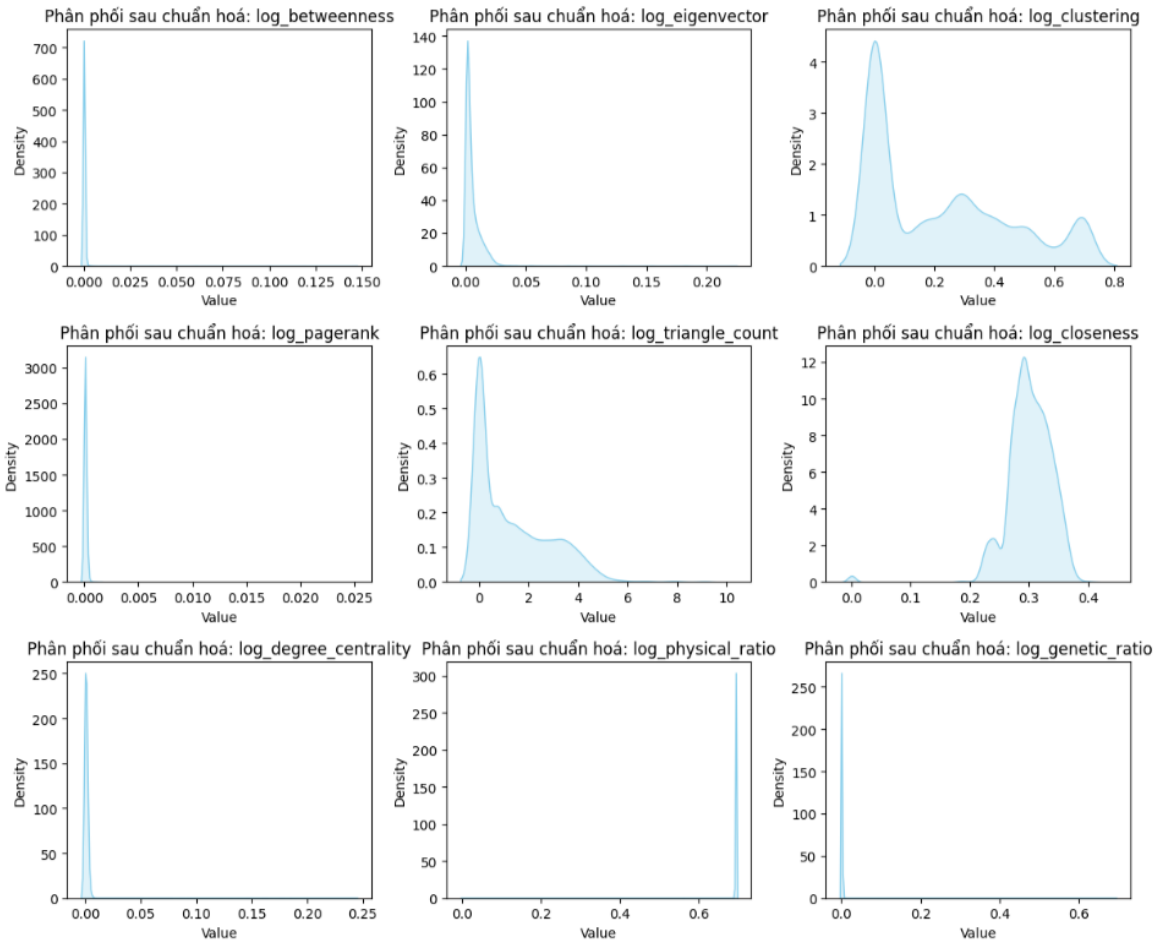
Hình 21 Phân phối các đặc trưng trước khi log_transform

- Ý nghĩa biểu đồ:

Biểu đồ thể hiện phân phối của các đặc trưng trong mạng protein, dưới dạng các biểu đồ mật độ. Các đặc trưng được thống kê gồm: betweenness, eigenvector, clustering, pagerank, triangle_count, closeness, degree_centrality, physical_ratio, genetic_ratio.

- Nhận xét:
  Các đặc trưng phân bố lệch mạnh về 0.
  - Betweenness, Pagerank, Degree Centrality, Eigenvector:
    - Phân phối lệch mạnh về 0, đa số giá trị nhỏ, chỉ một số ít node có giá trị cao → mạng có một số protein rất nổi bật, số còn lại kém kết nối.
  - Closeness Centrality:
    - Phân bố gần chuẩn, tập trung chủ yếu quanh 0.3–0.4 → các node trong mạng có khoảng cách trung bình tương đối gần nhau, thể hiện mạng liên thông chặt.
  - Clustering:
    - Phân bố đa cực: có nhiều đỉnh → tồn tại nhiều cụm con (community) với mức độ liên kết nội bộ khác nhau.
  - Triangle Count:
    - Phân bố cực kỳ lệch: nhiều node có triangle count bằng 0 hoặc rất nhỏ, một số có giá trị lớn bất thường → tồn tại vài node thuộc về các cụm siêu kết nối.
  - Physical Ratio & Genetic Ratio:
    - Phân bố chủ yếu tập trung ở gần 0 hoặc gần 1 → Mỗi tương tác chủ yếu là thuần vật lý hoặc thuần di truyền, rất ít tương tác lai.

Tiếp theo, tiến hành log_transform:



Hình 22 Phân phối đặc trưng sau log_transform

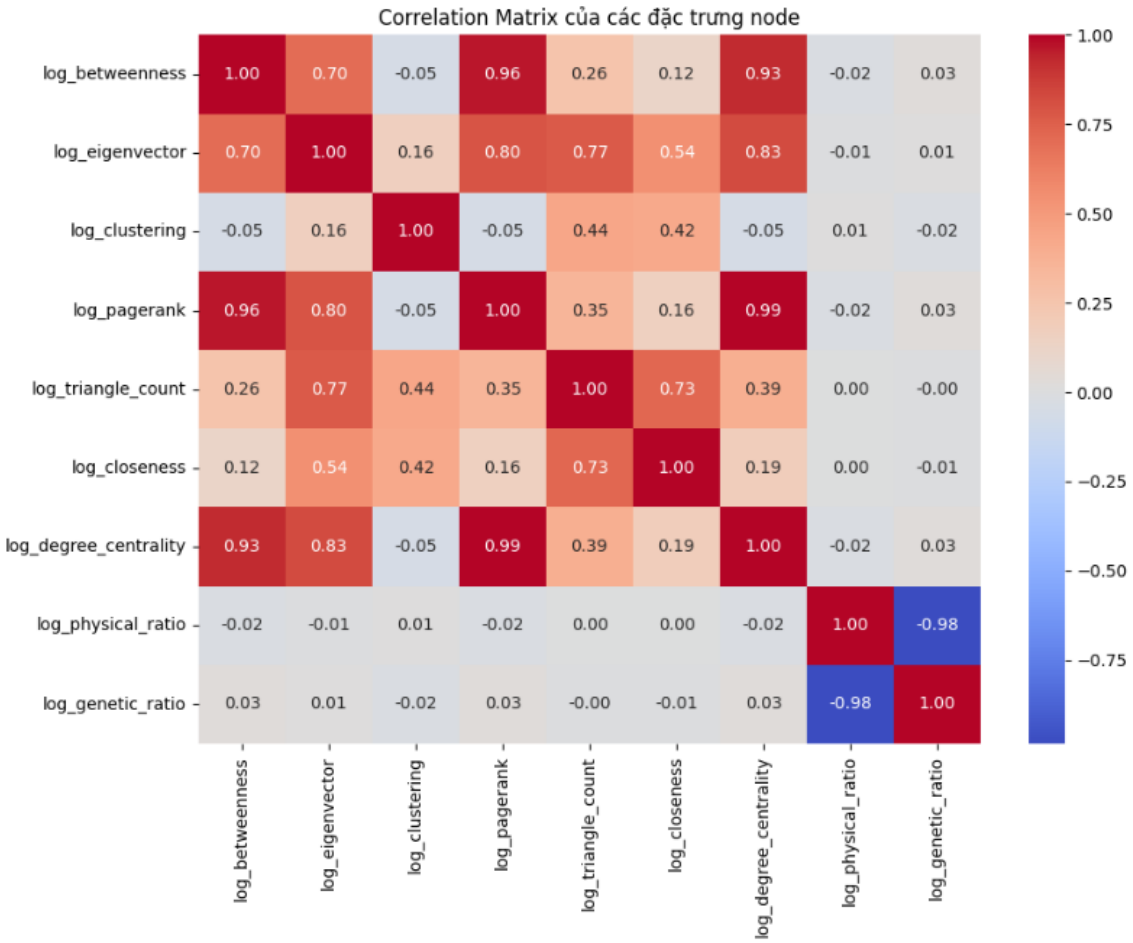
- Ý nghĩa biểu đồ:
  Phân phối sau chuẩn hóa log của các đặc trưng trong mạng. Mục tiêu là giảm ảnh hưởng của các giá trị ngoại lai (outliers), giúp phân phối dữ liệu trở nên gần chuẩn hơn, thuận lợi hơn cho các mô hình học máy.
- Nhận xét:
  - log_betweenness, log_pagerank, log_degree_centrality, log_eigenvector:
    - Giá trị vẫn phân bố lệch mạnh sau chuẩn hóa.
    - Sau log-transform, các giá trị vẫn bị dồn về phía 0, cho thấy phần lớn node có giá trị nhỏ.
  - log_closeness, log_clustering, log_triangle_count:
    - Phân phối cải thiện rõ sau log-transform.
    - log_closeness: Phân phối sau log gần chuẩn hơn, đỉnh rõ ràng hơn.
    - log_clustering: Trước có nhiều đỉnh (đa cực), sau log vẫn còn một chút đa đỉnh nhưng cải thiện độ trơn hơn.

        - log_triangle_count: Phân phối trước log rất lệch, nhiều giá trị cực lớn. Phân phối sau log trải rộng và trơn hơn.
    - log_physical_ratio, log_genetic_ratio:
        - Không thay đổi nhiều sau log.
        - Trước và sau log vẫn có phân phối nhọn cực đại tại hai đầu (0 và 1).

3.2.4. Lựa chọn đặc trưng

Tính ma trận tương quan của các đặc trưng. Đặt ngưỡng tương quan threshold = 0.9.

Duyệt qua ma trận tương quan để tìm đặc trưng cần loại bỏ - là những đặc trưng có độ tương quan vượt ngưỡng threshold.



Hình 23 Ma trận tương quan các đặc trưng đã tính

- Ý nghĩa biểu đồ:
  Biểu đồ là ma trận tương quan (Correlation Matrix) giữa các đặc trưng (features) đã được log-transform của các node trong một mạng lưới. Mỗi ô vuông biểu diễn hệ số tương quan Pearson giữa hai đặc trưng:
    - Giá trị +1: tương quan dương (hai đặc trưng cùng tăng/giảm).

    - Giá trị -1: tương quan âm (một tăng một giảm).
    - Giá trị 0: không có tương quan tuyến tính.

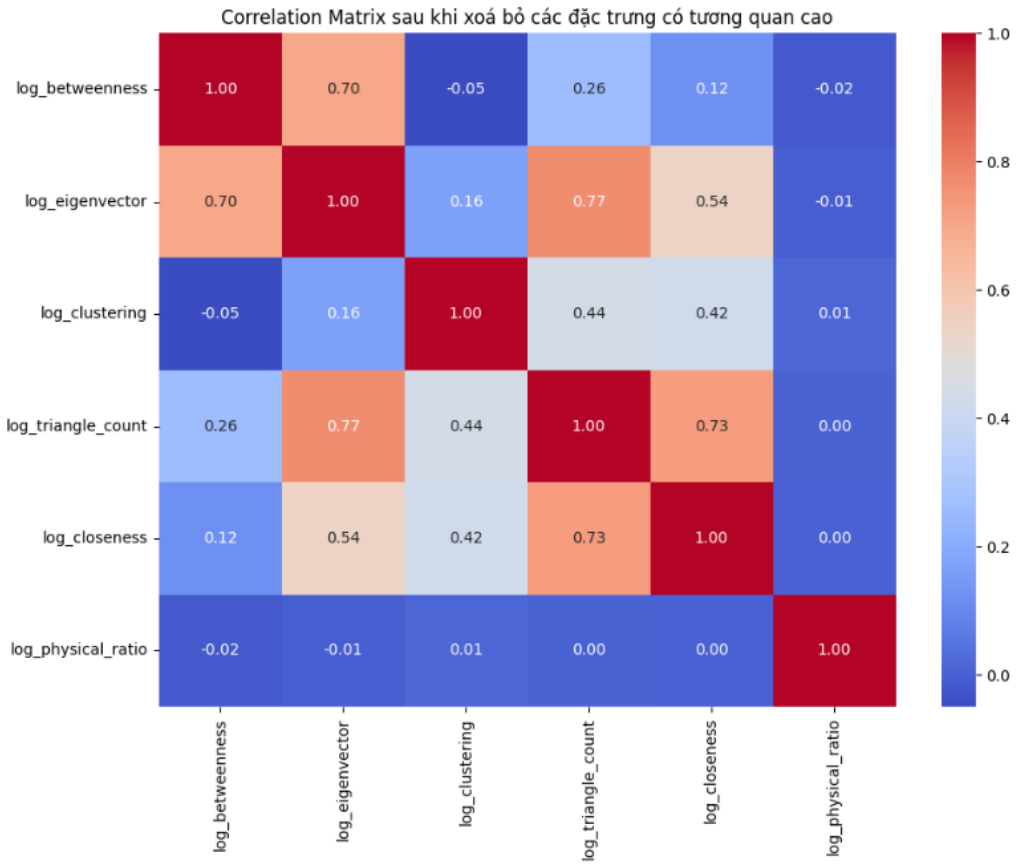
  Ma trận giúp xác định các cặp đặc trưng có liên hệ chặt chẽ, trùng lặp thông tin hoặc mang thông tin riêng biệt.

- Nhận xét:
    - Tương quan cao nhất:
        - log_degree_centrality và log_pagerank: r = 0.99.
        - log_betweenness và log_pagerank: r = 0.96.
        - log_betweenness và log_degree_centrality: r = 0.93.

        → Các đặc trưng này phản ánh tính trung tâm của node, và có thể dư thừa nếu dùng đồng thời trong mô hình.

    - Tương quan thấp hoặc gần 0:
        - log_physical_ratio và log_genetic_ratio với hầu hết đặc trưng mạng đều có r ≈ 0
          → Chúng mang thông tin độc lập, có thể giúp mô hình phân biệt tốt hơn.
    - Tương quan âm mạnh:
        - log_physical_ratio và log_genetic_ratio: r = -0.98

→ Hai đặc trưng này gần như là phản ánh đối lập nhau, chỉ nên giữ một trong phân tích chính.



Hình 24 Ma trận tương quan sau khi loại bỏ các đặc trưng có tương quan cao

- Ý nghĩa biểu đồ:
  Biểu đồ là ma trận tương quan (Correlation Matrix) giữa các đặc trưng node sau khi đã loại bỏ những đặc trưng có tương quan cao.
  - Mục tiêu: Giảm thiểu trùng lặp thông tin.
  - Mỗi ô thể hiện hệ số tương quan Pearson giữa hai đặc trưng đã log-transform.
  - Các ô càng gần 1 hoặc -1 thì hai đặc trưng càng liên hệ mạnh; gần 0 thì càng độc lập.
- Nhận xét:
  - Tổng quan: Các đặc trưng còn lại có tương quan thấp hoặc vừa phải với nhau → phù hợp cho phân tích đa biến.
  - Các cặp có tương quan trung bình:
    - log_triangle_count & log_closeness: r = 0.73.
    - log_triangle_count & log_eigenvector: r = 0.77.
      → Có thể cân nhắc giữ cả hai nếu không ảnh hưởng đến mô hình, hoặc loại bớt nếu muốn tăng tính phân biệt.
  - Đặc trưng độc lập rõ rệt:

        - log_physical_ratio có hệ số tương quan gần 0 với tất cả đặc trưng khác → giữ lại là hợp lý do mang thông tin riêng biệt.
    - Các đặc trưng đã bị loại bỏ trước đó (như pagerank, degree_centrality, genetic_ratio) là hợp lý vì trong biểu đồ trước, chúng có tương quan rất cao với nhau hoặc với các đặc trưng trung tâm khác.

3.2.5. Ghép và chuẩn hóa đặc trưng

Ghép các đặc trưng và gán cho x_nx, một ma trận đặc trưng đầu vào cho mô hình học sâu. Thực hiện chuẩn hóa đặc trưng theo công thức:

$$x' = \frac{x - \mu}{\sigma}$$

trong đó: μ là giá trị trung bình của cột, σ là độ lệch chuẩn của cột.

Sau khi đã thực hiện bước chuẩn hóa, tiến hành chuyển ma trận x_nx thành tensor PyTorch.

Lấy danh sách node trong đồ thị và sắp xếp lại. Tạo ánh xạ: ID node sang chỉ số index liên tục cho mỗi node. Chuyển danh sách cạnh sang dạng index, thay node gốc bằng index tương ứng. Sau đó tạo edge_index cho PyTorch Geometric.

3.2.6. Tạo danh sách cạnh và chia tập dữ liệu

Đầu tiên, thực hiện xáo trộn danh sách cạnh để đảm bảo chia tập train/validation/test ngẫu nhiên. Tiến hành chia tập train 70% (6957 cạnh), tập validation 15% (1490 cạnh) và tập test 15% (1490 cạnh). Tách riêng các tensor cạnh tương ứng với tập train, validation và test.

Thực hiện chuẩn hóa các tập cạnh, điều này rất quan trọng trong đồ thị vô hướng, vì (u, v) và (v, u) là cùng một cạnh, nên cần định dạng thống nhất để so sánh và lưu trữ không bị trùng lặp.

```python
# Chuẩn hóa cạnh
def normalize_edge(u, v):
    return tuple(sorted((u, v)))

# Lấy các tập cạnh
train_edges = set(normalize_edge(u, v) for u, v in data.train_pos_edge_index.t().cpu().numpy())
val_edges = set(normalize_edge(u, v) for u, v in data.val_pos_edge_index.t().cpu().numpy())
test_edges = set(normalize_edge(u, v) for u, v in data.test_pos_edge_index.t().cpu().numpy())

# Thống kê số lượng và trùng lặp
print(f"Số cạnh trong train set: {len(train_edges)}")
print(f"Số cạnh trong val set:   {len(val_edges)}")
print(f"Số cạnh trong test set:  {len(test_edges)}")

print(f"Số cạnh trùng giữa train và val:  {len(train_edges & val_edges)}")
print(f"Số cạnh trùng giữa train và test: {len(train_edges & test_edges)}")
print(f"Số cạnh trùng giữa val và test:   {len(val_edges & test_edges)}")
```

```
Số cạnh trong train set: 6957
Số cạnh trong val set:   1490
Số cạnh trong test set:  1490
Số cạnh trùng giữa train và val:  0
Số cạnh trùng giữa train và test: 0
Số cạnh trùng giữa val và test:   0
```

Hình 25 Đoạn mã minh hoạ việc tính toán các số cạnh

3.2.7. Xây dựng mô hình

3.2.7.1. Xây dựng GCN Encoder

Xây dựng lớp GCN Encoder để biến đổi đặc trưng của các node trong đồ thị dựa trên liên kết giữa chúng.

Hàm khởi tạo với các tham số gồm có số lượng đặc trưng đầu vào, danh sách số chiều của các hidden layer GCN - ở đây mô hình có 3 layer, mỗi layer 64 chiều. Tham số cuối cùng là xác suất dropout = 0.3, giúp giảm overfitting.

Hàm forward() với các tham số gồm có tensor đầu vào là đặc trưng của node và danh sách cạnh dùng để truyền thông tin giữa các node. Hàm trả về tensor x là embedding của các node, đã được học qua nhiều layer GCN.

Xây dựng hàm tính tích Hadamard giữa hai vector u và v để biến đặc trưng cặp node thành một vector duy nhất.

3.2.7.2. Xây dựng EdgeDecoder

Xây dựng lớp EdgeDecoder là một mô hình MLP (Multi-Layer Perceptron) dùng để dự đoán xác suất tồn tại cạnh giữa hai node trong đồ thị, dựa trên vector đặc trưng được trích từ cặp node đó.

Hàm khởi tạo có tham số là số chiều của vector đặc trưng đầu vào. Mô hình gồm 3 tầng fully-connected, được chèn giữa bởi ReLU và Dropout. Layer cuối là Sigmoid() để đưa output về dạng xác suất (0–1).

Hàm forward() có tham số là vector đặc trưng của cặp node, lấy từ hàm hadamard(). Hàm trả về xác suất tồn tại cạnh giữa 2 node.

3.2.7.3. Hàm huấn luyện

Hàm huấn luyện thực hiện huấn luyện mô hình GCN cho bài toán, đồng thời đánh giá hiệu suất trên tập validation sau mỗi epoch. Truyền vào hàm mô hình GCN Encoder, mô hình decoder và tập validation.

Đầu tiên, thực hiện đưa GCN Encoder và decoder vào chế độ huấn luyện, đồng thời xóa gradient cũ trước khi tính gradient mới trong vòng lặp. Kế đến lấy dữ liệu đầu vào là đặc trưng của các node và danh sách các cạnh dùng cho huấn luyện. Thực hiện chuẩn hóa embedding của mỗi node do GCN sinh ra.

Dự đoán giá trị xác suất các cạnh dương (là những cạnh có thật trong đồ thị) tồn tại bằng decoder. Tiếp tục tạo ra các cạnh âm (là những cạnh không tồn tại giữa hai node) với số lượng bằng cạnh dương sau đó dự đoán giá trị xác suất cạnh tồn tại bằng decoder. Gộp dự đoán cạnh dương và cạnh âm để tính binary cross entropy loss và AUC.

Cuối cùng, lan truyền ngược và cập nhật trọng số. Đánh giá mô hình trên tập validation. Hàm trả về kết quả là các chỉ số loss và AUC trên tập train và tập validation.

3.2.7.4. Hàm đánh giá

Hàm đánh giá được thiết kế để đánh giá mô hình GCN và Decoder sau khi đã huấn luyện. Mục tiêu là dự đoán xem cặp node có liên kết (edge) hay không, rồi đo lường hiệu suất bằng các chỉ số phân lớp loss, AUC, precision, recall, F1 và ma trận nhầm lẫn.

Truyền vào hàm mô hình GCN Encoder, mô hình decoder và tập validation. Đưa GCN Encoder và decoder vào chế độ đánh giá, đồng thời không cần tính gradient. Lấy dữ liệu đầu vào là đặc trưng của các node.

Dự đoán giá trị xác suất các cạnh dương (là những cạnh có thật trong đồ thị) tồn tại bằng decoder. Tiếp tục tạo ra các cạnh âm (là những cạnh không tồn tại giữa hai node) với số lượng bằng cạnh dương sau đó dự đoán giá trị xác suất cạnh tồn tại bằng decoder. Gộp dự đoán cạnh dương và cạnh âm.

Hàm trả về kết quả là các chỉ số loss, AUC, precision, recall, f1-score và ma trận nhầm lẫn.

3.2.8. Huấn luyện mô hình

Đầu tiên, tạo một mô hình GCNEncoder để trích xuất embedding (biểu diễn) cho các node. Tiếp tục tạo một mô hình Decoder dùng để dự đoán xác suất tồn tại cạnh giữa hai node. Sử dụng thuật toán Adam Optimizer để tối ưu mạng học sâu. Tối ưu cả Encoder lẫn Decoder bằng cách gộp parameters() của cả hai model.

Thực hiện gán hệ số learning rate = 0.001 và hệ số weight_decay = 1e-4.

Tiến hành tạo vòng lặp huấn luyện mô hình trong 50 epoch. Trong mỗi epoch, mô hình được huấn luyện và đánh giá hiệu suất trên tập validation, đồng thời lưu lại các chỉ số để theo dõi quá trình huấn luyện.

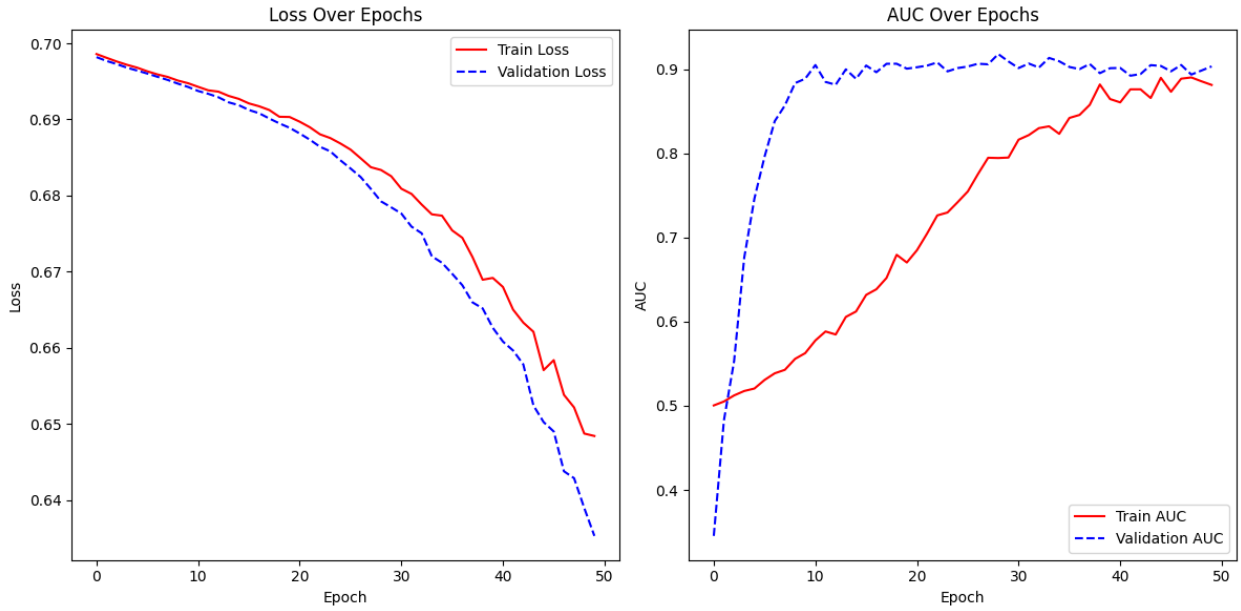
3.2.9. Dự đoán liên kết

Tiến hành lấy ngẫu nhiên danh sách 10 cặp node chưa có liên kết với nhau. Xây dựng hàm dự đoán liên kết có tham số là các mô hình đã được huấn luyện. Hàm dự đoán trả về một bảng các cặp node cùng với xác suất tồn tại liên kết ứng với từng cặp.

3.3. Trực quan kết quả

3.3.1. Loss và AUC

Sau khi huấn luyện 50 epochs, ta thu được kết quả:



Hình 26 Loss và AUC của 50 epochs

- Ý nghĩa biểu đồ:
  Biểu đồ gồm 2 phần:
  - Trái – Loss Over Epochs: Thể hiện sự thay đổi của hàm mất mát (Loss) trên tập huấn luyện và tập validation theo số epoch.
  - Phải – AUC Over Epochs: Thể hiện sự thay đổi của chỉ số AUC (Area Under Curve) trong quá trình huấn luyện và kiểm tra mô hình.

  Cả hai đồ thị giúp đánh giá quá trình học của mô hình theo thời gian (epoch).
- Nhận xét:
  - Loss:
    - Loss của cả train và validation đều giảm dần → mô hình đang học hiệu quả.

        - Validation loss thấp hơn train loss ở nhiều thời điểm.
        - Không thấy dấu hiệu overfitting rõ rệt.
    - AUC:
        - Validation AUC tăng rất nhanh, đạt giá trị cao (trên 0.9) chỉ sau ~10 epochs → mô hình phân biệt tốt trên tập validation sớm.
        - Train AUC tăng từ từ nhưng vẫn tiến bộ rõ ràng.
        - Đường AUC validation dao động nhẹ nhưng ổn định → mô hình tổng quát tốt.

3.3.2. Kết quả evaluation và Confusion Matrix

Bảng 13 Thống kê kết quả evaluation cuối cùng

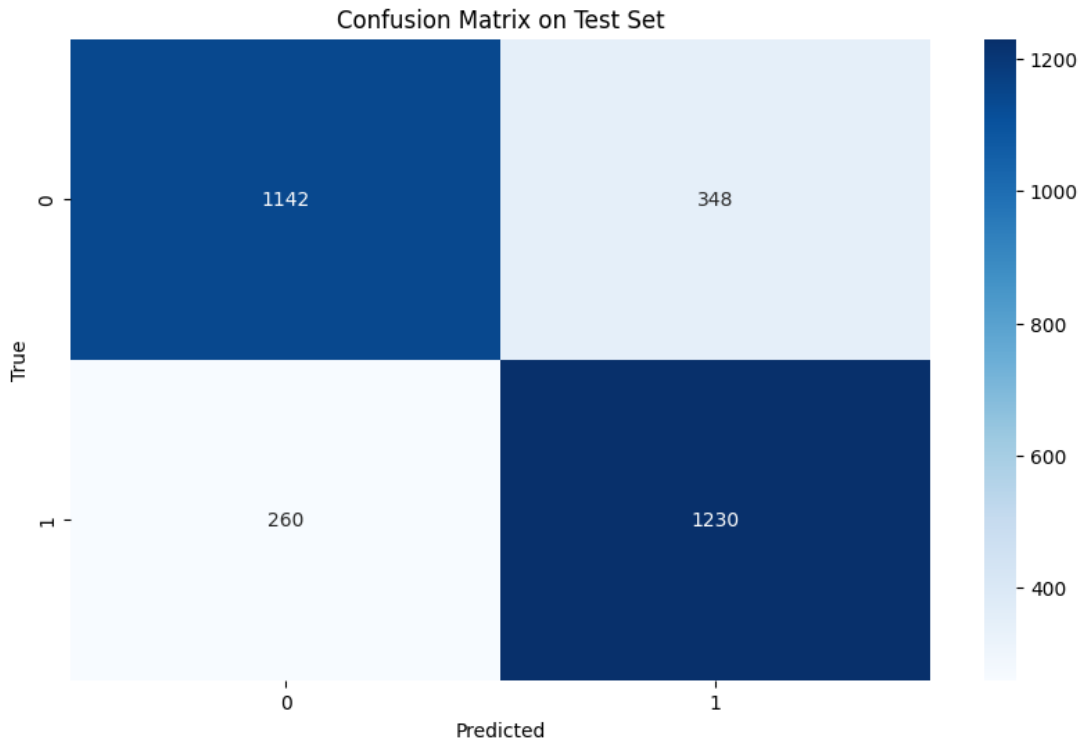
Train Loss	0.6484
Train AUC	0.8811
Test Loss	0.6356
Test AUC	0.8972
Test Precision	0.7795
Test Recall	0.8255
Test F1-score	0.8018

Test loss thấp hơn một chút so với train loss có thể do tập train có nhiều nhiễu hơn.

Không có dấu hiệu overfitting nghiêm trọng, vì test loss không cao hơn train loss.

AUC cao (> 0.88) ở cả train và test → mô hình có khả năng phân biệt tốt giữa liên kết có và không có.

Test AUC cao hơn train AUC một chút → điều này là tốt và cho thấy mô hình không học quá mức từ tập huấn luyện và có khả năng tổng quát hóa mạnh.

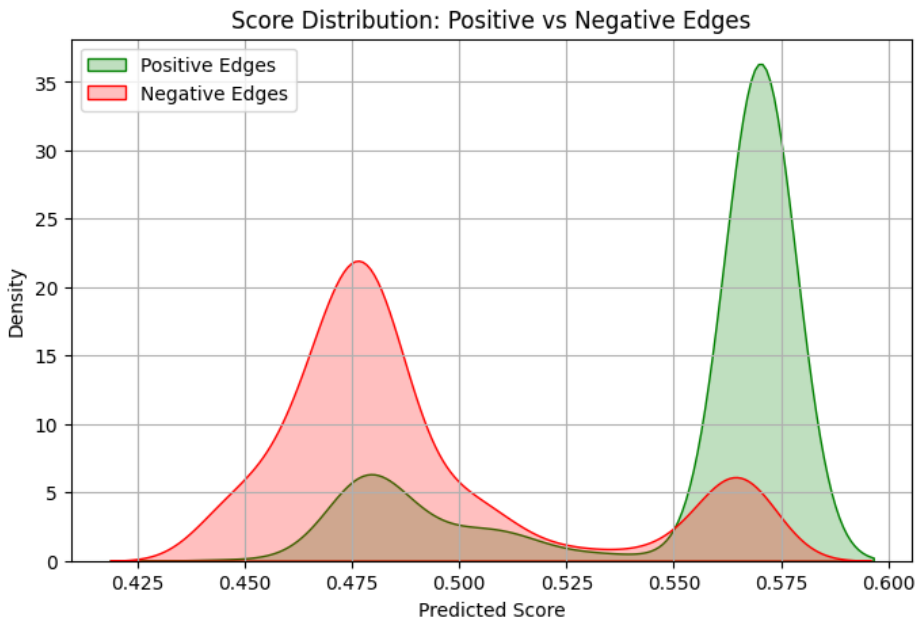


Hình 27 Confusion matrix

Ý nghĩa biểu đồ:
Ma trận nhầm lẫn thể hiện kết quả dự đoán của mô hình so với nhãn thực tế trên tập kiểm tra. Trong đó:
Trục tung (True): là nhãn thực tế.
Trục hoành (Predicted): là nhãn mô hình dự đoán.
Các ô thể hiện số lượng mẫu rơi vào từng trường hợp dự đoán đúng hoặc sai.
Nhận xét:
Mô hình hoạt động khá tốt với tỉ lệ dự đoán đúng cao ở cả hai lớp.

Tuy nhiên, có 348 mẫu dương tính giả (FP) và 260 mẫu âm tính giả (FN), nên vẫn có không ít nhầm lẫn – đặc biệt nếu ứng dụng đòi hỏi độ nhạy cao.

3.3.3. Phân phối điểm số giữa cạnh dương và cạnh âm



Hình 28 Phân phối điểm số giữa cạnh dương và cạnh âm

- Ý nghĩa biểu đồ:
  Biểu đồ thể hiện phân phối mật độ xác suất của điểm dự đoán cho hai loại cạnh trong bài toán link prediction:
  - Đường màu xanh (Positive Edges): điểm dự đoán của các cạnh thực sự tồn tại trong đồ thị.
  - Đường màu đỏ (Negative Edges): điểm dự đoán của các cạnh không tồn tại trong đồ thị.
  - Trục hoành là giá trị điểm dự đoán (Predicted Score), trục tung là mật độ (Density).
- Nhận xét:
  - Phân phối của Positive Edges tập trung mạnh ở khoảng [0.55, 0.58], cho thấy mô hình gán điểm cao cho các cạnh thật.
  - Ngược lại, Negative Edges tập trung ở khoảng [0.46, 0.48], điểm thấp hơn rõ rệt.
  - Hai phân phối có ít vùng chồng lấn, cho thấy khả năng phân biệt tốt giữa cạnh thật và cạnh giả.
  - Điều này cho thấy mô hình có hiệu suất dự đoán tốt, phù hợp với các chỉ số cao đã thể hiện ở biểu đồ AUC và confusion matrix trước đó.

3.3.4. Dự đoán tương tác 10 cặp protein ngẫu nhiên

Bảng 14 Dự đoán tương tác 10 cặp protein ngẫu nhiên

ID A	Protein A	ID B	Protein B	Score
107432	CDC34	108360	EPHA8	0.487804
121271	MFF	123894	PRAM1	0.486641
137505	C9orf173	112286	SDC4	0.482402
108811	FZD2	110050	KRT5	0.480510
107720	COX4I1	107790	CRMP1	0.473541
109708	IGFBP4	118942	GNL2	0.461644
117101	SNAPIN	124975	BPIFB1	0.460870
107683	COL4A4	124370	ZFYVE19	0.457185
131495	PDDC1	106842	APOC3	0.445987
137505	C9orf173	109708	IGFBP4	0.443155

TÀI LIỆU THAM KHẢO

[1] L. Liu et al., “Integrating Sequence and Network Information to Enhance Protein-Protein Interaction Prediction Using Graph Convolutional Networks,” in 2019 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Oct. 2019, pp. 1762–1768. doi: 10.1109/BIBM47256.2019.8983330.